DATA 260: Agentic AI

Instructor: Prof. Dr. Simon Shim
Student Name: Trapti Kulshrestha
STUDENT ID: 019124833
Homework: 6

Section 0: Personal Configuration:

SID4	4833
PORT_BASE:	8333
PREFIX	s4833
SEED	4833
VERIFY_SEED	264833
DOMAIN_ID:	1
Assigned Domain	Clinical Trial Listings

Hardware: Apple MacBook Pro

Local Model: qwen3:8b (Ollama); embeddings: nomic-embed-text

GitHub Repository: https://github.com/Psycoder0611/data260-4833

Project Setup:

For Homework 6, I continued working in my existing DATA 260 repository and created the hw6-work branch to keep the new implementation separate from the submitted HW5 state.

I extended the existing FastAPI application instead of creating a separate project. For HW6, I added a MongoDB document API for clinical trials, a chat memory system with short-term, long-term, and episodic memory, a memory cost experiment, and a file-based memory system using AGENT.md and MEMORY.md.

MongoDB runs locally in Docker. The new code is in the hw6 package and in routers/hw6_trials.py and routers/hw6_chat.py. I only added two import lines and two include_router lines to main.py, so the HW2 to HW5 code and routes were not changed. I also created reports/hw06/ with a raw/ folder for the run log, metrics, raw experiment files, probes, AI use notes, and verification evidence.

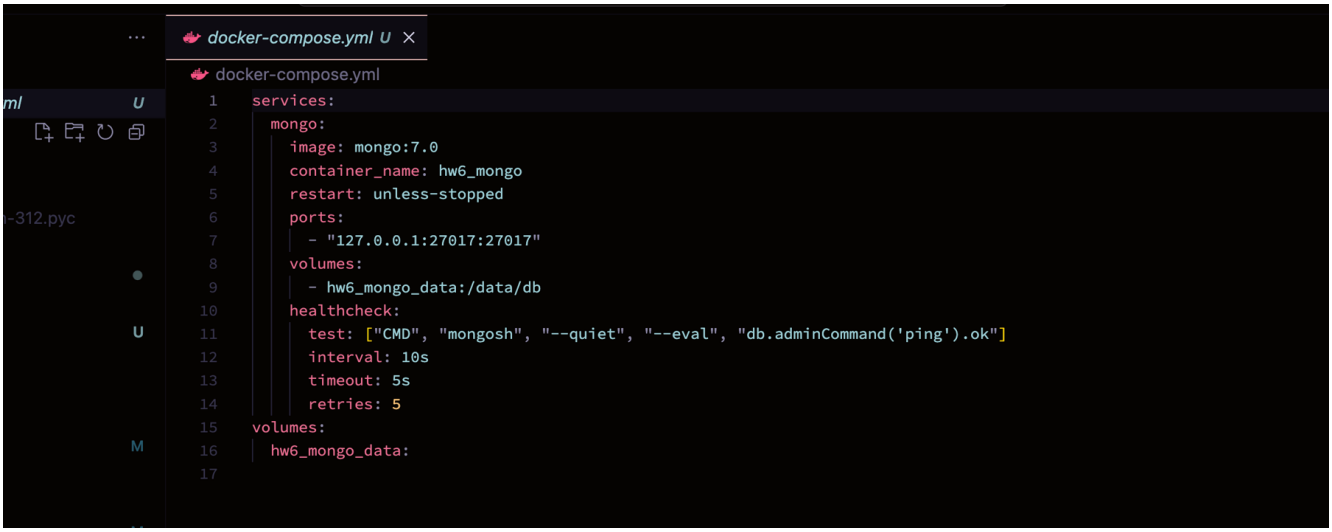
Part 1 – MongoDB Document API

1.1 Database Setup:

I used a local MongoDB 7 container started with Docker Compose, so the project runs without a cloud account or password. The container listens only on localhost. The database is s4833_doc and the main collection is clinical_trials. A setup script creates the collection with validation rules and indexes.

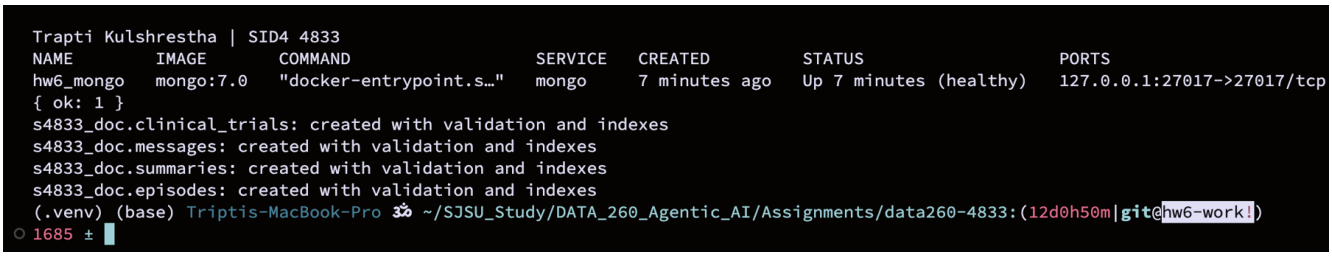
Code 1 - Docker Compose file (docker-compose.yml)

MongoDB 7 container, localhost port, data volume and health check.



Output 1 - Container health and database setup

The container is healthy, answers a ping, and init_hw6_mongo.py creates the collections.



1.2 MongoDB Connection and Server Setup:

The server settings are in hw6/config.py and the PyMongo connection is in hw6/mongo.py. The connection string is read from the environment with a local default, so no credentials are stored in the code. main.py already had CORS middleware, and FastAPI reads JSON request bodies on its own. I only registered the two new HW6 routers there. The application runs on port 8333.

Output 2 - FastAPI running on port 8333

Uvicorn starts the application on the assigned port.



1.3 Pydantic Schema:

I defined the trial document with Pydantic. The title is required and limited to 100 characters. The description is optional. Status is limited to recruiting, active, completed or terminated and defaults to recruiting. Phase is limited to phase1, phase2, phase3, phase4 or not_applicable. The scheduled date must be a valid date and time. Extra fields are rejected. createdAt and updatedAt are set by the server.

Field	Type	Rule
trialTitle	string	required, 1-100 characters
description	string / null	optional, up to 2000 characters
status	enum	recruiting (default), active, completed, terminated
phase	enum	phase1, phase2, phase3, phase4, not_applicable
scheduledDate	datetime	required, valid date and time
createdAt / updatedAt	datetime	set automatically by the API

Code 2 - Pydantic schema (hw6/schemas.py)

Create schema with required title, enums, default status and datetime.

```
10
11  class TrialCreate(BaseModel):
12      """Body for POST /api/hw6/trials."""
13
14      model_config = ConfigDict(str_strip_whitespace=True, extra="forbid")
15
16      trialTitle: str = Field(..., min_length=1, max_length=100)
17      description: Optional[str] = Field(default=None, max_length=2000)
18      status: StatusEnum = "recruiting"
19      phase: PhaseEnum
20      scheduledDate: datetime
21
```

1.4 Create Trial:

The POST endpoint saves a new trial and returns it with its id and both timestamps. A successful request returns HTTP 201, and the default status is applied when I leave it out.

Code 3 - POST and GET list /api/hw6/trials (routers/hw6_trials.py)

Create inserts the document with createdAt and updatedAt; list filters by status and phase, newest first.

```python
@router.post("", response_model=TrialOut, status_code=status.HTTP_201_CREATED)
def create_trial(payload: TrialCreate):
    """Create a trial document with automatic createdAt and updatedAt."""
    document = payload.model_dump()
    stamp = now_utc()
    document["createdAt"] = stamp
    document["updatedAt"] = stamp
    try:
        result = trials_collection().insert_one(document)
        created = trials_collection().find_one({"_id": result.inserted_id})
        return serialize_trial(created)
    except PyMongoError as exc:
        raise db_error(exc, "creating the trial")


@router.get("", response_model=List[TrialOut])
def list_trials(
    trial_status: Optional[StatusEnum] = Query(default=None, alias="status"),
    phase: Optional[PhaseEnum] = None,
    limit: int = Query(default=100, ge=1, le=500),
):
    """Return trials, newest first. Optional filters: status, phase."""
    query = {}
    if trial_status:
        query["status"] = trial_status
    if phase:
```

Output 3 - Create trial

HTTP 201 with the new id, timestamps and the default status.

```
Trapti Kulshrestha | SID4 4833
Student: Trapti Kulshrestha | SID4 4833 | HW6 demo
2026-10-16 22:31:30

--- POST /api/hw6/trials (create) -> HTTP 201
{
  "id": "6ad30833c61dc01c82dacf6d",
  "trialTitle": "Glucose Monitoring Study",
  "description": "Study evaluating glucose tracking in adults.",
  "status": "recruiting",
  "phase": "phase2",
  "scheduledDate": "2026-10-15T10:00:00Z",
  "createdAt": "2026-10-17T05:31:31.018000Z",
  "updatedAt": "2026-10-17T05:31:31.018000Z"
}
```

1.5 Retrieve Trials:

The GET endpoint returns all trials, newest first. It can also filter by status or phase.

Output 4 - List trials

Both trials returned, newest first.

```
--- GET /api/hw6/trials (all) -> HTTP 200
[
  {
    "id": "6ad30833c61dc01c82dacf6e",
    "trialTitle": "Hypertension Drug Trial",
    "description": "Study evaluating glucose tracking in adults.",
    "status": "active",
    "phase": "phase3",
    "scheduledDate": "2026-10-15T10:00:00Z",
    "createdAt": "2026-10-17T05:31:31.031000Z",
    "updatedAt": "2026-10-17T05:31:31.031000Z"
  },
  {
    "id": "6ad30833c61dc01c82dacf6d",
    "trialTitle": "Glucose Monitoring Study",
    "description": "Study evaluating glucose tracking in adults.",
    "status": "recruiting",
    "phase": "phase2",
    "scheduledDate": "2026-10-15T10:00:00Z",
    "createdAt": "2026-10-17T05:31:31.018000Z",
    "updatedAt": "2026-10-17T05:31:31.018000Z"
  }
]
```

1.6 Retrieve Trial by ID:

The GET-by-ID endpoint returns one trial. A badly formed id returns HTTP 400. A valid id that does not exist returns HTTP 404 with the message Trial not found.

Code 4 - GET, PUT and DELETE /api/hw6/trials/{id} (routers/hw6_trials.py)

Id check, lookup, partial update with find_one_and_update, delete, and 404 handling.

```python
@router.get("/{trial_id}", response_model=TrialOut)
def get_trial(trial_id: str):
    """Return one trial by id."""
    check_id(trial_id)
    try:
        doc = trials_collection().find_one({"_id": ObjectId(trial_id)})
    except PyMongoError as exc:
        raise db_error(exc, "reading the trial")
    if doc is None:
        raise HTTPException(404, "Trial not found.")
    return serialize_trial(doc)


@router.put("/{trial_id}", response_model=TrialOut)
def update_trial(trial_id: str, payload: TrialUpdate):
    """Update the fields that were sent and refresh updatedAt."""
    check_id(trial_id)
    changes = payload.model_dump(exclude_unset=True)
    if not changes:
        raise HTTPException(400, "No fields provided to update.")
    changes["updatedAt"] = now_utc()
    try:
        doc = trials_collection().find_one_and_update(
            {"_id": ObjectId(trial_id)},
            {"$set": changes},
            return_document=ReturnDocument.AFTER,
        )
    except PyMongoError as exc:
        raise db_error(exc, "updating the trial")
    if doc is None:
        raise HTTPException(404, "Trial not found.")
    return serialize_trial(doc)


@router.delete("/{trial_id}", status_code=status.HTTP_204_NO_CONTENT)
def delete_trial(trial_id: str):
    """Delete one trial by id."""
    check_id(trial_id)
    try:
        result = trials_collection().delete_one({"_id": ObjectId(trial_id)})
    except PyMongoError as exc:
        raise db_error(exc, "deleting the trial")
    if result.deleted_count == 0:
        raise HTTPException(404, "Trial not found.")
    return Response(status_code=status.HTTP_204_NO_CONTENT)

```

Output 5 - Get trial by id

The trial is returned for a valid id; the bad id and unknown id blocks show 400 and 404.

```
--- GET /api/hw6/trials/6ad30833c61dc01c82dacf6d -> HTTP 200
{
  "id": "6ad30833c61dc01c82dacf6d",
  "trialTitle": "Glucose Monitoring Study",
  "description": "Study evaluating glucose tracking in adults.",
  "status": "recruiting",
  "phase": "phase2",
  "scheduledDate": "2026-10-15T10:00:00Z",
  "createdAt": "2026-10-17T05:31:31.018000Z",
  "updatedAt": "2026-10-17T05:31:31.018000Z"
}
```

```
--- Bad id format -> HTTP 400
{
  "detail": "Invalid trial id format."
}

--- Unknown id -> HTTP 404
{
  "detail": "Trial not found."
}
```

1.7 Update Trial:

The PUT endpoint changes only the fields I send and refreshes updatedAt. If no fields are sent, it returns HTTP 400.

Output 6 - Update trial

Status and phase changed; updatedAt is newer than createdAt.

```
--- PUT (status -> active, phase -> phase3) -> HTTP 200
{
  "id": "6ad30833c61dc01c82dacf6d",
  "trialTitle": "Glucose Monitoring Study",
  "description": "Study evaluating glucose tracking in adults.",
  "status": "active",
  "phase": "phase3",
  "scheduledDate": "2026-10-15T10:00:00Z",
  "createdAt": "2026-10-17T05:31:31.018000Z",
  "updatedAt": "2026-10-17T05:31:31.040000Z"
}
```

1.8 Delete Trial:

The DELETE endpoint removes the trial and returns HTTP 204. Asking for the same trial again returns HTTP 404.

Output 7 - Delete trial

HTTP 204, then HTTP 404 for the deleted id.

```
--- DELETE /api/hw6/trials/6ad30833c61dc01c82dacf6d -> HTTP 204

--- GET deleted trial -> HTTP 404
{
  "detail": "Trial not found."
}
```

1.9 Validation Tests:

I tested invalid data to check the validation. Pydantic rejects bad input before it reaches MongoDB and returns HTTP 422 with the field name and the reason. MongoDB also has its own $jsonSchema rules, so bad data is blocked in two places. Database problems return HTTP 500, or 503 when MongoDB is not reachable.

Test	Result
Missing trialTitle	422
Invalid status value	422
Title longer than 100 characters	422
Wrong date type	422
Badly formed id	400
Unknown id	404

Output 8 - Missing required field

The request without trialTitle is rejected with HTTP 422.


```
--- Validation: missing trialTitle -> HTTP 422
{
  "detail": [
    {
      "type": "missing",
      "loc": [
        "body",
        "trialTitle"
      ],
      "msg": "Field required",
      "input": {
        "description": "Study evaluating glucose tracking in adults.",
        "status": "recruiting",
        "phase": "phase2",
        "scheduledDate": "2026-10-15T10:00:00"
      }
    }
  ]
}
```


Output 9 - Invalid enum value

status = paused is rejected with HTTP 422.


```
--- Validation: invalid status enum -> HTTP 422
{
  "detail": [
    {
      "type": "literal_error",
      "loc": [
        "body",
        "status"
      ],
      "msg": "Input should be 'recruiting', 'active', 'completed' or 'terminated'",
      "input": "paused",
      "ctx": {
        "expected": "'recruiting', 'active', 'completed' or 'terminated'"
      }
    }
  ]
}
```

```
--- Validation: title longer than 100 characters -> HTTP 422
{
  "detail": [
    {
      "type": "string_too_long",
      "loc": [
        "body",
        "trialTitle"
      ],
      "msg": "String should have at most 100 characters",
      "input": "xxxx
```

Part 2 – Short-, Long-, and Episodic Memory

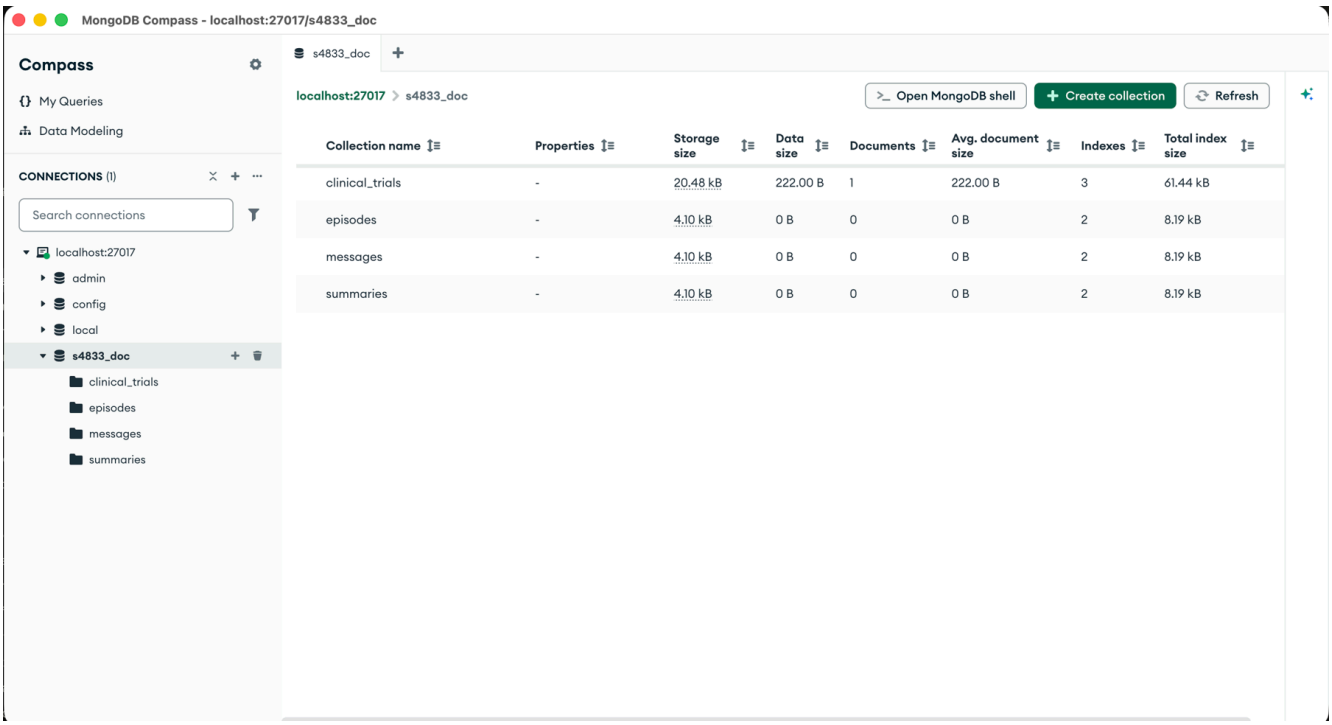
Part 2 uses the same database, s4833_doc, with three new collections: messages, summaries and episodes. The Part 1 collection was not changed. The model is qwen3:8b through my HW1 model adapter, and embeddings come from nomic-embed-text.

2.1 MongoDB Memory Collections:

Collection	Fields
messages	user_id, session_id, role (user or assistant), content, created_at
summaries	user_id, session_id (null for lifetime), scope (session or user), text, created_at
episodes	user_id, session_id, fact, importance (0 to 1), embedding (array of floats), created_at

Output 1 - Memory collections created

The memory collections exist in s4833_doc.



2.2 POST /api/chat:

Each chat request follows the same steps. The server saves the user message, loads the short-term window, loads the latest session and lifetime summaries, retrieves the top episodic facts, builds the prompt, calls the local model, and saves the reply. After the reply, it extracts new facts and updates the summaries when the schedule says so. The response contains the reply and an object that shows which memory was used.

Code 1 - Chat, memory and aggregate endpoints (routers/hw6_chat.py)

The router exposes the chat, memory and aggregate endpoints for the MongoDB memory system.

```python
@router.post("/api/chat")
def chat(request: ChatRequest):
    """Save the message, build the memory prompt, call the local model, save the reply."""
    cfg = memory.MemoryConfig()
    if request.short_term_n is not None:
        cfg.short_term_n = request.short_term_n
    if request.memory_policy is not None:
        cfg.policy = request.memory_policy
    try:
        return memory.chat_turn(
            request.user_id,
            request.session_id,
            request.message,
            cfg,
            persist=request.persist,
        )
    except llm.ModelUnavailableError as exc:
        raise HTTPException(503, f"{exc}. Is Ollama running with {config.MODEL_NAME}?")
    except PyMongoError as exc:
        raise mongo_problem(exc)


@router.get("/api/memory/{user_id}")
def memory_view(user_id: str):
    """Return recent messages, latest summaries and recent episodic facts."""
    try:
        return memory.get_memory(user_id)
    except PyMongoError as exc:
        raise mongo_problem(exc)


@router.get("/api/aggregate/{user_id}")
def aggregate_view(user_id: str):
    """Return daily message counts and the most recent summaries."""
    try:
        return memory.get_aggregate(user_id)
    except PyMongoError as exc:
        raise mongo_problem(exc)
```

Code 2 - Prompt assembly (hw6/memory.py)

System primer, summaries, short-term window, current message and episodic facts.

```python
def build_messages(
    message: str,
    window: List[Dict[str, Any]],
    session_summary: Optional[str],
    lifetime_summary: Optional[str],
    episodes: List[Dict[str, Any]],
    digest: str = "",
) -> Tuple[List[Dict[str, str]], Dict[str, str]]:
    """Assemble the model prompt and the text of each part (for token accounting)."""
    long_term = ""
    if lifetime_summary:
        long_term += f"\n\n## Lifetime summary\n{lifetime_summary}"
    if session_summary:
        long_term += f"\n\n## Session summary\n{session_summary}"
    short_term_note = f"\n\n## Earlier in this chat (condensed)\n{digest}" if digest else ""
    system_text = SYSTEM_PRIMER + long_term + short_term_note
    episodic = ""
    if episodes:
        facts = "\n".join(f"- {item['fact']}" for item in episodes)
        episodic = f"\n\n(Remembered facts that may help:\n{facts})"
    messages = [{"role": "system", "content": system_text}]
    messages += [{"role": m["role"], "content": m["content"]} for m in window]
    messages.append({"role": "user", "content": message + episodic})
    parts = {
        "primer": SYSTEM_PRIMER,
        "long_term": long_term.strip(),
        "short_term": "\n".join(
            [short_term_note.strip()] + [f"{m['role']}: {m['content']}" for m in window]
        ).strip(),
        "episodic": episodic.strip(),
        "current": message,
    }
    return messages, parts
```

2.3 Short-Term Memory:

Short-term memory is a sliding window of the last N messages in the session. The message being answered is not part of the window; it is added at the end. The default N is 12.

Code 3 - Short-term window (hw6/memory.py)

Last N messages of the session, oldest first.

```python
def load_short_term(user_id: str, session_id: str, n: int, exclude_id: Any = None) -> List[Dict[str, Any]]:
    """Return the last n messages of a session in time order (optionally skipping one id)."""
    cursor = (
        get_collection(config.MESSAGES_COLLECTION)
        .find({"user_id": user_id, "session_id": session_id})
        .sort([("created_at", -1), ("_id", -1)])
        .limit(n + 1)
    )
    docs = [doc for doc in cursor if exclude_id is None or doc["_id"] != exclude_id]
    docs = docs[:n]
    docs.reverse()
    return docs
```

(variable) cursor: Cursor

2.4 Long-Term Summaries:

After every 5th user message, the model writes a short bullet summary of the session from the old summary and the newest messages. After every 3 session summaries, they are condensed into one lifetime summary with scope user and no session id.

Code 4 - Session summary (hw6/memory.py)

Previous summary plus new messages become the new session summary.

```python
def summarize_session(user_id: str, session_id: str, cfg: MemoryConfig, usage: Dict[str, Any]) -> str:
    """Write a new session summary from the previous one plus the latest messages."""
    recent = load_short_term(user_id, session_id, cfg.summarize_every * 2)
    previous = latest_summary(user_id, "session", session_id)
    transcript = "\n".join(f"{m['role']}: {m['content']}" for m in recent)
    body = (
        f"Previous summary:\n{previous['text'] if previous else '(none yet)'}\n\n"
        f"New messages:\n{transcript}"
    )
    result = llm.chat(
        [
            {"role": "system", "content": SUMMARY_PROMPT},
            {"role": "user", "content": body},
        ]
    )
    add_aux(usage, result)
    save_summary(user_id, session_id, "session", result["content"])
    return result["content"]
```

2.5 Episodic Memory:

For every user message, the model picks up to 3 short facts and gives each an importance score from 0 to 1. Each fact is turned into a vector with nomic-embed-text and stored in MongoDB as an array of floats. For a new message, I embed it, compare it with the stored facts using cosine similarity, and put the top 3 facts into the prompt. The similarity is computed in Python, so no vector database is needed.

Code 5 - Embedding and retrieval (hw6/memory.py)

Cosine similarity between the message and stored facts; top-k returned.

```python
239
240 def retrieve_episodes(user_id: str, query: str, top_k: int) -> List[Dict[str, Any]]:
241     """Return the top_k stored facts closest (cosine) to the query text."""
242     docs = list(get_collection(config.EPISODES_COLLECTION).find({"user_id": user_id}))
243     if not docs or top_k <= 0:
244         return []
245     try:
246         query_vector = llm.embed(query)
247     except llm.ModelUnavailableError as exc:
248         logger.warning("Episodic retrieval skipped: %s", exc)
249         return []
250     scored = []
251     for doc in docs:
252         vector = doc.get("embedding") or []
253         if len(vector) != len(query_vector):
254             continue
255         scored.append(
256             {
257                 "fact": doc["fact"],
258                 "importance": doc.get("importance", 0.5),
259                 "score": round(cosine_similarity(query_vector, vector), 4),
260             }
261         )
262     scored.sort(key=lambda item: item["score"], reverse=True)
263     return scored[:top_k]
```

2.6 GET /api/memory/{user_id}:

This endpoint returns the last 16 messages, the latest session summary, the latest lifetime summary and the last 20 episodic facts. The long embedding arrays are left out of the response.

2.7 GET /api/aggregate/{user_id}:

This endpoint returns daily message counts grouped by date with a MongoDB aggregation, and the most recent summaries.

Code 6 - Aggregation (hw6/memory.py)

$group by date and a list of recent summaries.

```
461
462  def get_aggregate(user_id: str, recent_summaries: int = 5) -> Dict[str, Any]:
463      """Return daily message counts (MongoDB aggregation) and the newest summaries."""
464      pipeline = [
465          {"$match": {"user_id": user_id}},
466          {
467              "$group": {
468                  "_id": {"$dateToString": {"format": "%Y-%m-%d", "date": "$created_at"}},
469                  "messages": {"$sum": 1},
470                  "user_messages": {"$sum": {"$cond": [{"$eq": ["$role", "user"]}, 1, 0]}},
471              }
472          },
473          {"$sort": {"_id": 1}},
474      ]
475      daily = [
476          {"date": row["_id"], "messages": row["messages"], "user_messages": row["user_messages"]}
477          for row in get_collection(config.MESSAGES_COLLECTION).aggregate(pipeline)
478      ]
479      summaries = list(
480          get_collection(config.SUMMARIES_COLLECTION)
481          .find({"user_id": user_id})
482          .sort([("created_at", -1), ("_id", -1)])
483          .limit(recent_summaries)
484      )
485      return {
486          "user_id": user_id,
487          "daily_message_counts": daily,
488          "recent_summaries": [clean_doc(doc) for doc in summaries],
```

2.8 10-Message Test:

I sent 11 messages in one session. The output shows the window growing, summaries appearing at messages 5 and 10, facts being stored, and the last question being answered with the help of a retrieved fact.

Output 2 - Chat session, first messages

Replies from the assistant and the memory line after each turn.

```
Trapti Kulshrestha | SID4 4833
Student: Trapti Kulshrestha | SID4 4833 | HW6 demo
2026-10-16 23:03:08
user_id=demo_230308 session_id=session1

[1] you: Hi! I'm Dana, a field ecologist living in Monterey.
    assistant: Hello, Dana! It's a pleasure to meet you. I hope you're enjoying your work in Monterey's rich ecological areas!
    memory: short-term=0 | session summary=no | lifetime=no | episodic retrieved=0
    stored facts: ['Dana is a field ecologist.', 'Dana lives in Monterey.']

[2] you: I'm planning a 12-day trip to Iceland starting March 3rd.
    assistant: That sounds exciting, Dana! Iceland is a fantastic place for nature exploration, and your background as an ecologist will mak
e your trip even more rewarding. Enjoy your adventure!
    memory: short-term=2 | session summary=no | lifetime=no | episodic retrieved=2
    retrieved facts: ['Dana is a field ecologist.', 'Dana lives in Monterey.']
    stored facts: ['User is planning a 12-day trip to Iceland.', 'Trip starts on March 3rd.']

[3] you: My cat Biscuit will stay with my neighbour Omar.
    assistant: That's great to hear! Biscuit will be in good hands with Omar. Safe travels, Dana! 🌍🐾
    memory: short-term=4 | session summary=no | lifetime=no | episodic retrieved=3
    retrieved facts: ['Dana lives in Monterey.', 'Dana is a field ecologist.', 'User is planning a 12-day trip to Iceland.']
    stored facts: ["User's cat Biscuit will stay with neighbour Omar."]

[4] you: What should I pack for cold, wet weather?
    assistant: For cold, wet weather in Iceland, pack waterproof clothing, thermal layers, sturdy boots, and a reusable water bottle. Don't
forget a hat and gloves! 🧥
    memory: short-term=6 | session summary=no | lifetime=no | episodic retrieved=3
    retrieved facts: ['User is planning a 12-day trip to Iceland.', 'Dana is a field ecologist.', 'Trip starts on March 3rd.']

[5] you: I'm vegetarian, so food planning matters too.
    assistant: That's a good point! For vegetarian food planning in Iceland, look for vegan-friendly cafes, bring some snacks, and check out
 local markets for fresh options. Enjoy your trip! 🌿🌍
    memory: short-term=8 | session summary=no | lifetime=no | episodic retrieved=3
    retrieved facts: ['Dana is a field ecologist.', "User's cat Biscuit will stay with neighbour Omar.", 'User is planning a 12-day trip to
Iceland.']
    stored facts: ['User is vegetarian.']
    >> session summary created

[6] you: My camera is a Nikon Z6 with a 400mm lens.
    assistant: That's a great setup for wildlife photography! The Nikon Z6 with a 400mm lens will work well in Iceland's rugged landscapes.
Don't forget to protect your gear from the weather! 📸❄️
```

Output 3 - Chat session, later messages

Summaries created, facts retrieved and the final recall question.

```
[6] you: My camera is a Nikon Z6 with a 400mm lens.
    assistant: That's a great setup for wildlife photography! The Nikon Z6 with a 400mm lens will work well in Iceland's rugged landscapes.
Don't forget to protect your gear from the weather! 📸❄️
    memory: short-term=10 | session summary=yes | lifetime=no | episodic retrieved=3
    retrieved facts: ['Dana is a field ecologist.', 'Trip starts on March 3rd.', 'User is vegetarian.']
    stored facts: ['User owns a Nikon Z6 camera with a 400mm lens.']

[7] you: I'm also writing a paper on seabird nesting, due April 20th.
    assistant: That's fantastic! Iceland is a prime location for seabird studies. Make sure to document your observations carefully—good luc
k with your paper, and enjoy your research trip! 🐦📚
    memory: short-term=12 | session summary=yes | lifetime=no | episodic retrieved=3
    retrieved facts: ['User is planning a 12-day trip to Iceland.', 'Dana is a field ecologist.', 'Trip starts on March 3rd.']
    stored facts: ['User is writing a paper on seabird nesting.', 'Paper is due on April 20th.']

[8] you: My co-author is Dr. Lena Okafor.
    assistant: That's great to know! Dr. Lena Okafor will be a great collaborator on your seabird nesting research. Good luck with your pape
r—due April 20th! 🐦📚
    memory: short-term=12 | session summary=yes | lifetime=no | episodic retrieved=3
    retrieved facts: ['User is writing a paper on seabird nesting.', 'Dana is a field ecologist.', 'Paper is due on April 20th.']
    stored facts: ["User's co-author is Dr. Lena Okafor."]

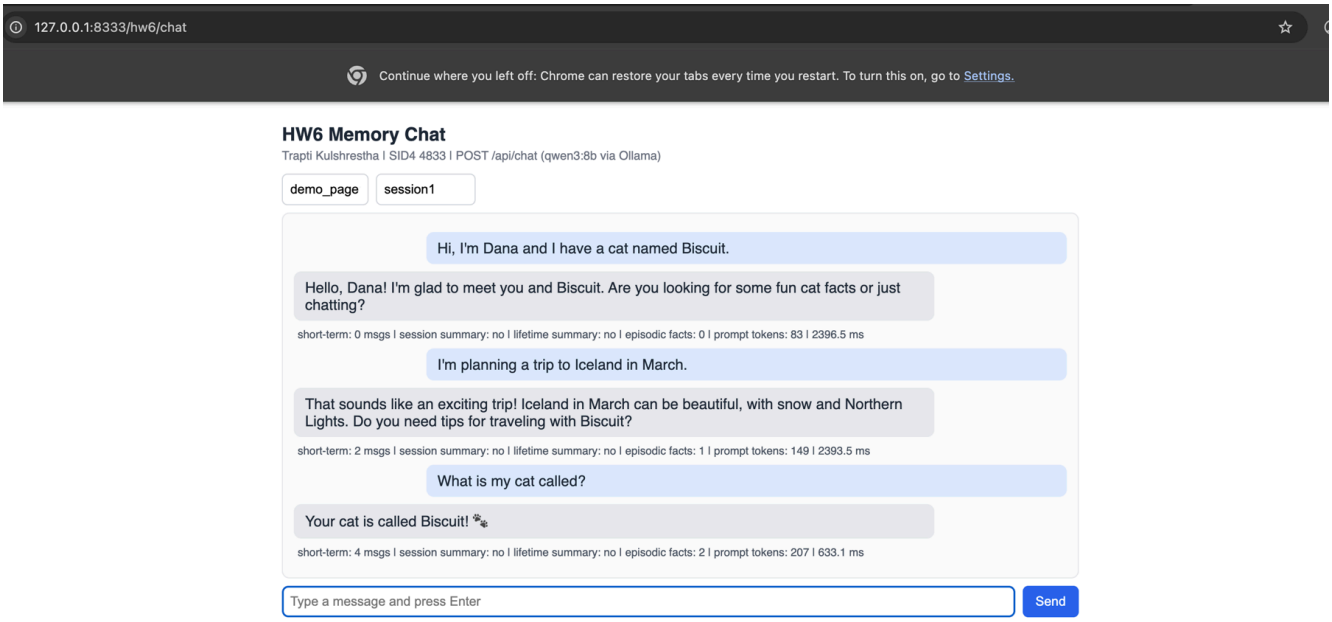
[9] you: Any tip for staying focused while writing?
    assistant: Break your writing into small, manageable tasks and set daily goals. Stay consistent, and don't hesitate to reach out to Dr.
Okafor for feedback—good luck with your paper! 📝📚
    memory: short-term=12 | session summary=yes | lifetime=no | episodic retrieved=3
    retrieved facts: ['User is writing a paper on seabird nesting.', "User's co-author is Dr. Lena Okafor.", 'Paper is due on April 20th.']

[10] you: I usually write between 6 and 9 am.
    assistant: That's a great time to write! Stick to your routine, and use the early morning quiet to focus on your seabird nesting paper.
Good luck with your deadline on April 20th! 📖🐦
    memory: short-term=12 | session summary=yes | lifetime=no | episodic retrieved=3
    retrieved facts: ['Paper is due on April 20th.', 'User is writing a paper on seabird nesting.', "User's co-author is Dr. Lena Okafor."]
    stored facts: ['User writes between 6 and 9 am.']
    >> session summary created

[11] you: Quick check: what is my cat called and who looks after her?
    assistant: Your cat is called Biscuit, and she'll be staying with your neighbor Omar while you're in Iceland. 🐱
    memory: short-term=12 | session summary=yes | lifetime=no | episodic retrieved=3
    retrieved facts: ["User's cat Biscuit will stay with neighbour Omar.", "User's co-author is Dr. Lena Okafor.", 'User is writing a paper
on seabird nesting.']
```

Output 4 - Conversation in the chat page

The small HTML page calling POST /api/chat.



Output 5 - GET /api/memory/{user_id}

Messages, summaries and episodic facts for the demo user.


```
--- GET /api/memory/demo_230308
{
  "user_id": "demo_230308",
  "messages": [
    {
      "id": "6ad30fa6bce85feeb05178f8",
      "user_id": "demo_230308",
      "session_id": "session1",
      "role": "user",
      "content": "What should I pack for cold, wet weather?",
      "created_at": "2026-10-17T06:03:18.687000+00:00"
    },
    {
      "id": "6ad30fa8bce85feeb05178f9",
      "user_id": "demo_230308",
      "session_id": "session1",
      "role": "assistant",
      "content": "For cold, wet weather in Iceland, pack waterproof clothing, thermal layers, sturdy boots, and a reusable water bottle. Don
\u2019t forget a hat and gloves! \ud83c\udf28\ufe0f\ud83e\udde6",
      "created_at": "2026-10-17T06:03:20.391000+00:00"
    },
    {
      "id": "6ad30fa8bce85feeb05178fa",
      "user_id": "demo_230308",
      "session_id": "session1",
      "role": "user",
      "content": "I'm vegetarian, so food planning matters too.",
      "created_at": "2026-10-17T06:03:20.659000+00:00"
    },
    {
      "id": "6ad30faabce85feeb05178fb",
      "user_id": "demo_230308",
      "session_id": "session1",
      "role": "assistant",
      "content": "That\u2019s a good point! For vegetarian food planning in Iceland, look for vegan-friendly cafes, bring some snacks, and c
heck out local markets for fresh options. Enjoy your trip! \ud83c\udf3f\ud83c\udf0d",
      "created_at": "2026-10-17T06:03:22.368000+00:00"
    },
    {
      "id": "6ad30faebce85feeb05178fe",
```

```
      "user_id": "demo_230308",
      "session_id": "session1",
      "role": "assistant",
      "content": "That\u2019s a great time to write! Stick to your routine, and use the early morning quiet to focus on your seabird nesting
 paper. Good luck with your deadline on April 20th! \ud83d\udcd6\ud83d\udc26",
      "created_at": "2026-10-17T06:03:43.803000+00:00"
    },
    {
      "id": "6ad30fc5bce85feeb051790e",
      "user_id": "demo_230308",
      "session_id": "session1",
      "role": "user",
      "content": "Quick check: what is my cat called and who looks after her?",
      "created_at": "2026-10-17T06:03:49.222000+00:00"
    },
    {
      "id": "6ad30fc7bce85feeb051790f",
      "user_id": "demo_230308",
      "session_id": "session1",
      "role": "assistant",
      "content": "Your cat is called Biscuit, and she'll be staying with your neighbor Omar while you're in Iceland. \ud83d\udc31",
      "created_at": "2026-10-17T06:03:51.118000+00:00"
    }
  ],
  "session_summary": {
    "id": "6ad30fc5bce85feeb051790d",
    "user_id": "demo_230308",
    "session_id": "session1",
    "scope": "session",
    "text": "- Dana is a field ecologist living in Monterey.  \n- She is planning a 12-day trip to Iceland starting March 3rd.  \n- Her cat
Biscuit will stay with neighbour Omar.  \n- She is asking about packing for cold, wet weather.  \n- She is vegetarian and is concerned about
 food planning.  \n- She is using a Nikon Z6 with a 400mm lens for wildlife photography.",
    "created_at": "2026-10-17T06:03:49.211000+00:00"
  },
  "lifetime_summary": null,
  "episodes
```


```
(.venv) (base) Triptis-MacBook-Pro 🐼 ~/SJSU_Study/DATA_260_Agentic_AI/Assignments/data260-4833:(12d1h28m|git@hw6-work!)
1695 ± echo "Trapti Kulshrestha | SID4 4833"
curl -s http://127.0.0.1:8333/api/memory/demo_230308 | python -c "import sys,json; d=json.load(sys.stdin); print(json.dumps({'lifetime_summa
ry': d['lifetime_summary'], 'episodes': d['episodes']}, indent=2))"
Trapti Kulshrestha | SID4 4833
{
  "lifetime_summary": null,
  "episodes": [
    {
      "id": "6ad30fc9bce85feeb0517911",
      "user_id": "demo_230308",
      "session_id": "session1",
      "fact": "Someone looks after the user's cat.",
      "importance": 0.8,
      "created_at": "2026-10-17T06:03:53.063000+00:00"
    },
    {
      "id": "6ad30fc9bce85feeb0517910",
      "user_id": "demo_230308",
      "session_id": "session1",
      "fact": "User's cat is called by a specific name.",
      "importance": 0.9,
      "created_at": "2026-10-17T06:03:53.038000+00:00"
    },
    {
      "id": "6ad30fc0bce85feeb051790c",
      "user_id": "demo_230308",
      "session_id": "session1",
      "fact": "User writes between 6 and 9 am.",
      "importance": 0.7,
      "created_at": "2026-10-17T06:03:44.992000+00:00"
    },
    {
      "id": "6ad30fb9bce85feeb0517907",
      "user_id": "demo_230308",
      "session_id": "session1",
      "fact": "User's co-author is Dr. Lena Okafor.",
      "importance": 1.0,
      "created_at": "2026-10-17T06:03:37.905000+00:00"
    },
    {
```

```
    "id": "6ad30fb6bce85feeb0517904",
    "user_id": "demo_230308",
    "session_id": "session1",
    "fact": "Paper is due on April 20th.",
    "importance": 0.8,
    "created_at": "2026-10-17T06:03:34.150000+00:00"
  },
  {
    "id": "6ad30fb6bce85feeb0517903",
    "user_id": "demo_230308",
    "session_id": "session1",
    "fact": "User is writing a paper on seabird nesting.",
    "importance": 0.9,
    "created_at": "2026-10-17T06:03:34.125000+00:00"
  },
  {
    "id": "6ad30fb2bce85feeb0517900",
    "user_id": "demo_230308",
    "session_id": "session1",
    "fact": "User owns a Nikon Z6 camera with a 400mm lens.",
    "importance": 0.9,
    "created_at": "2026-10-17T06:03:30.345000+00:00"
  },
  {
    "id": "6ad30fabbce85feeb05178fc",
    "user_id": "demo_230308",
    "session_id": "session1",
    "fact": "User is vegetarian.",
    "importance": 0.9,
    "created_at": "2026-10-17T06:03:23.325000+00:00"
  },
  {
    "id": "6ad30fa6bce85feeb05178f7",
    "user_id": "demo_230308",
    "session_id": "session1",
    "fact": "User's cat Biscuit will stay with neighbour Omar.",
    "importance": 0.9,
```


Output 6 - GET /api/aggregate/{user_id}

Daily message counts and recent summaries.


```
  food planning.  \n- She is using a Nikon Z6 with a 400mm lens for wildlife photography.",
     "created_at": "2026-10-17T06:03:49.211000+00:00"
   },
   "lifetime_summary": null,
   "episodes

 --- GET /api/aggregate/demo_230308
 {
   "user_id": "demo_230308",
   "daily_message_counts": [
     {
       "date": "2026-10-17",
       "messages": 22,
       "user_messages": 11
     }
   ],
   "recent_summaries": [
     {
       "id": "6ad30fc5bce85feeb051790d",
       "user_id": "demo_230308",
       "session_id": "session1",
       "scope": "session",
       "text": "- Dana is a field ecologist living in Monterey.  \n- She is planning a 12-day trip to Iceland starting March 3rd.  \n- Her ca
 t Biscuit will stay with neighbour Omar.  \n- She is asking about packing for cold, wet weather.  \n- She is vegetarian and is concerned abo
 ut food planning.  \n- She is using a Nikon Z6 with a 400mm lens for wildlife photography.",
       "created_at": "2026-10-17T06:03:49.211000+00:00"
     },
     {
       "id": "6ad30faebce85feeb05178fd",
       "user_id": "demo_230308",
       "session_id": "session1",
       "scope": "session",
       "text": "- Dana is a field ecologist living in Monterey.  \n- She is planning a 12-day trip to Iceland starting March 3rd.  \n- Her ca
 t Biscuit will stay with neighbour Omar.  \n- She is asking about packing for cold, wet weather.  \n- She is vegetarian and is concerned abo
 ut food planning.",
       "created_at": "2026-10-17T06:03:26.373000+00:00"
     }
   ]
 }
 (.venv) (base) Triptis-MacBook-Pro 🐼 ~/SJSU_Study/DATA_260_Agentic_AI/Assignments/data260-4833:(12d1h28m|git@hw6-work!)
○ 1695 ±
```


- Short-term: the window grows by two messages per turn (0, 2, 4, ...) until it reaches the limit of 12. With N = 4 it stops at 4 (turns_n12.csv and turns_n4.csv in reports/hw06/raw/).

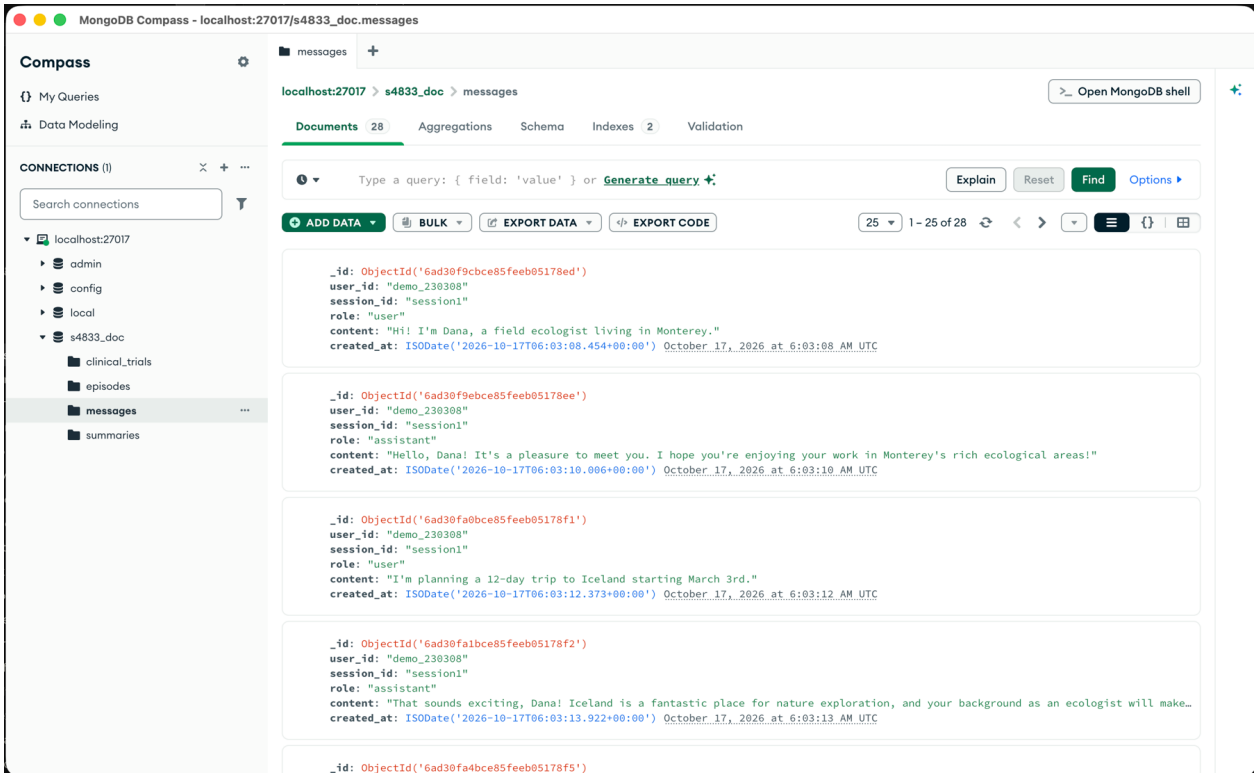
- Long-term: a session summary is written after every 5th user message and used from turn 6. The lifetime summary is used from turn 16, after three session summaries (visible in the 30-message runs; the shorter demo above creates two session summaries).
- Episodic: facts are stored from the user's statements and the right facts are retrieved for the recall question, as the retrieved-facts line in the chat session above shows.

2.9 MongoDB Memory Evidence:

MongoDB Compass shows documents in the three memory collections for the demo user.

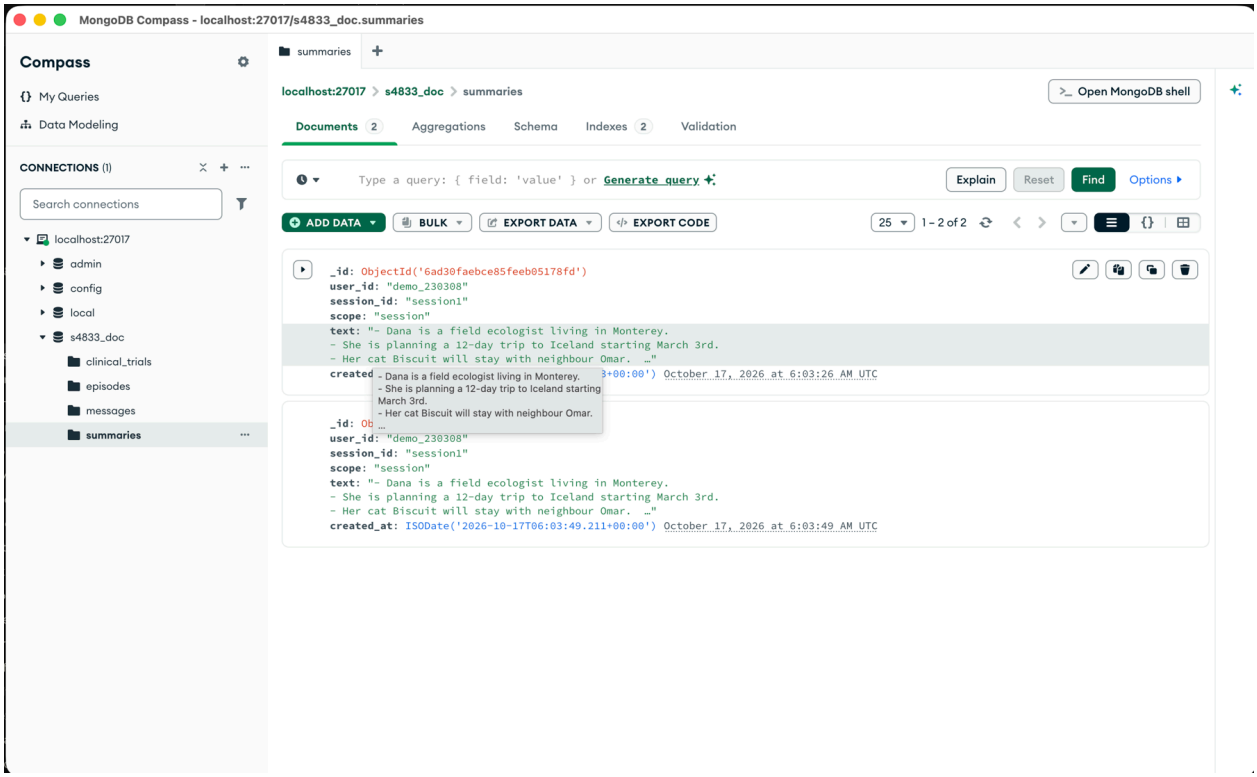
Output 7 - messages collection (Compass)

Stored user and assistant messages with session_id and created_at.



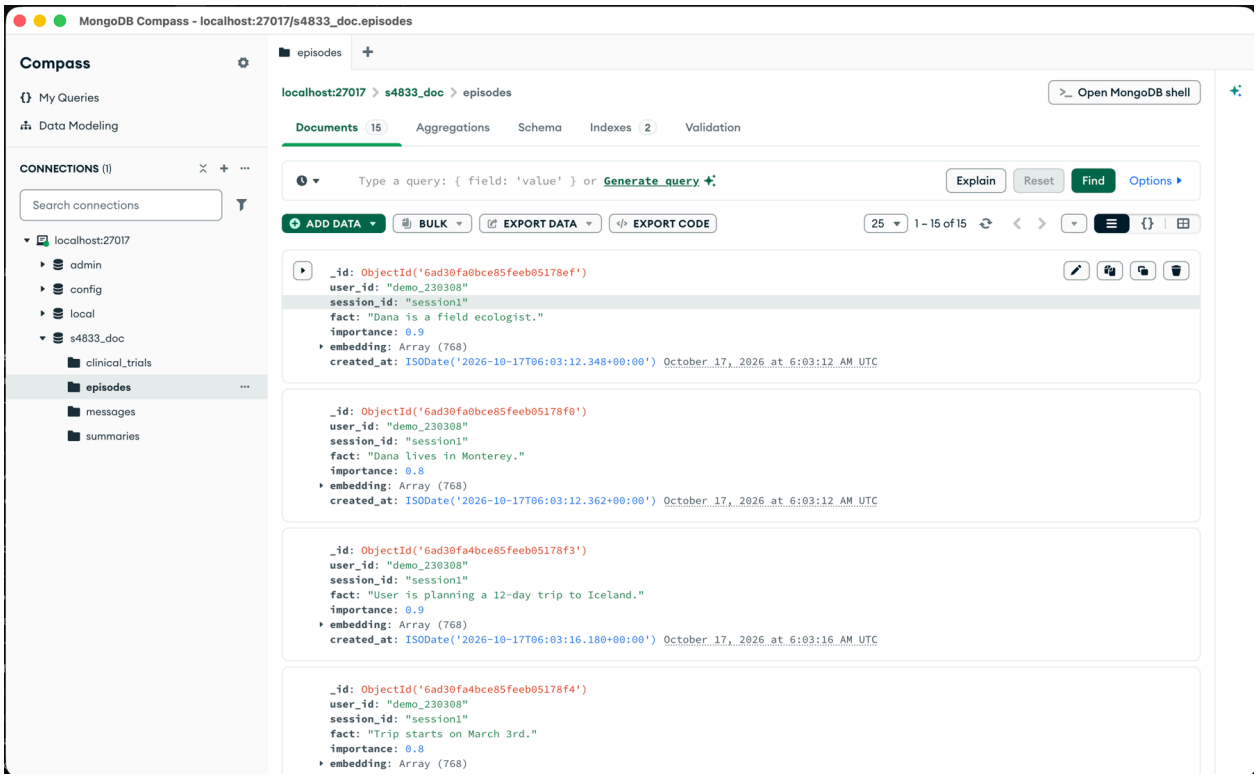
Output 8 - summaries collection (Compass)

Session summaries and a lifetime summary (scope user, session_id null).



Output 9 - episodes collection (Compass)

Facts with importance and an embedding array.



Part 3 – What Memory Costs

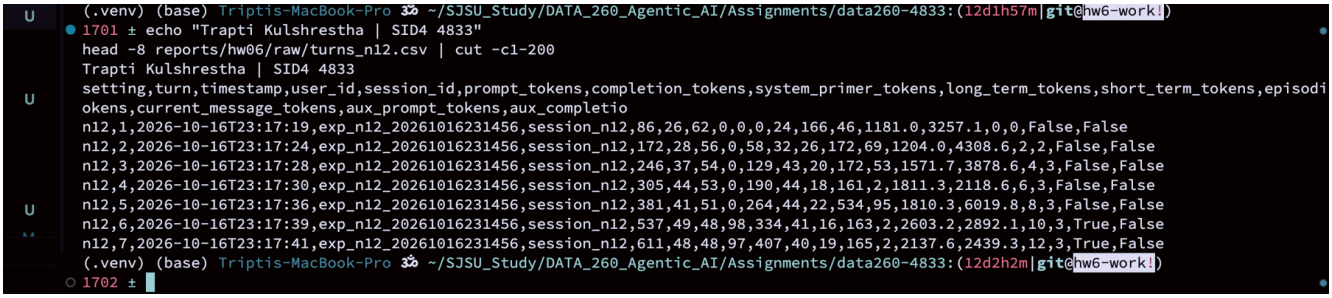
I ran the same fixed 30-message conversation (data/hw6_conversation_30.json) in a fresh user and session for each setting. After each run I asked the same five memory probes. The probes were written before any graded run and were not changed after the results.

3.1 Token Logging:

Every /api/chat response includes a usage object with prompt tokens, completion tokens, the tokens in each part of the prompt, and the request latency. Ollama only reports the total prompt size, so I split it across the parts in proportion to their text length. Tokens used by background calls (fact extraction and summaries) are logged separately. The experiment script saves one row per request in reports/hw06/raw/turns_*.csv.

Output 1 - Raw per-turn data (turns_n12.csv)

Prompt and completion tokens, tokens per part and latency for each turn.

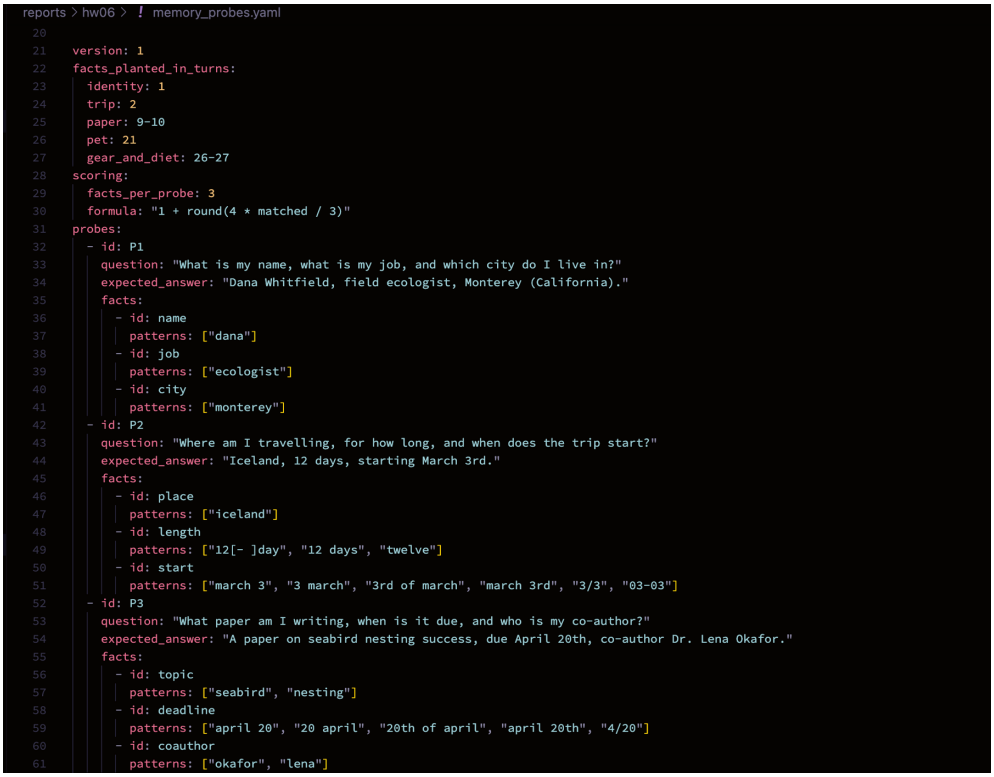


3.2 memory_probes.yaml:

The file has five fixed questions. Each has an expected answer, three required facts and a 1 to 5 score. A reply gets 1 + round(4 × matched facts / 3), so 3 of 3 facts gives 5, 2 of 3 gives 4, 1 of 3 gives 2, and none gives 1. The facts were planted at different points of the conversation, so some are far back and some are recent.

Code 1 - memory_probes.yaml

Five questions, expected facts and the scoring rule.



3.3 Short-Term Memory Experiments (N = 4, 12, 30):

The same 30-message conversation was run with SHORT_TERM_N set to 4, 12 and 30. The run command writes real console output with timestamps to RUN_LOG.txt.

Output 2 - Experiment run

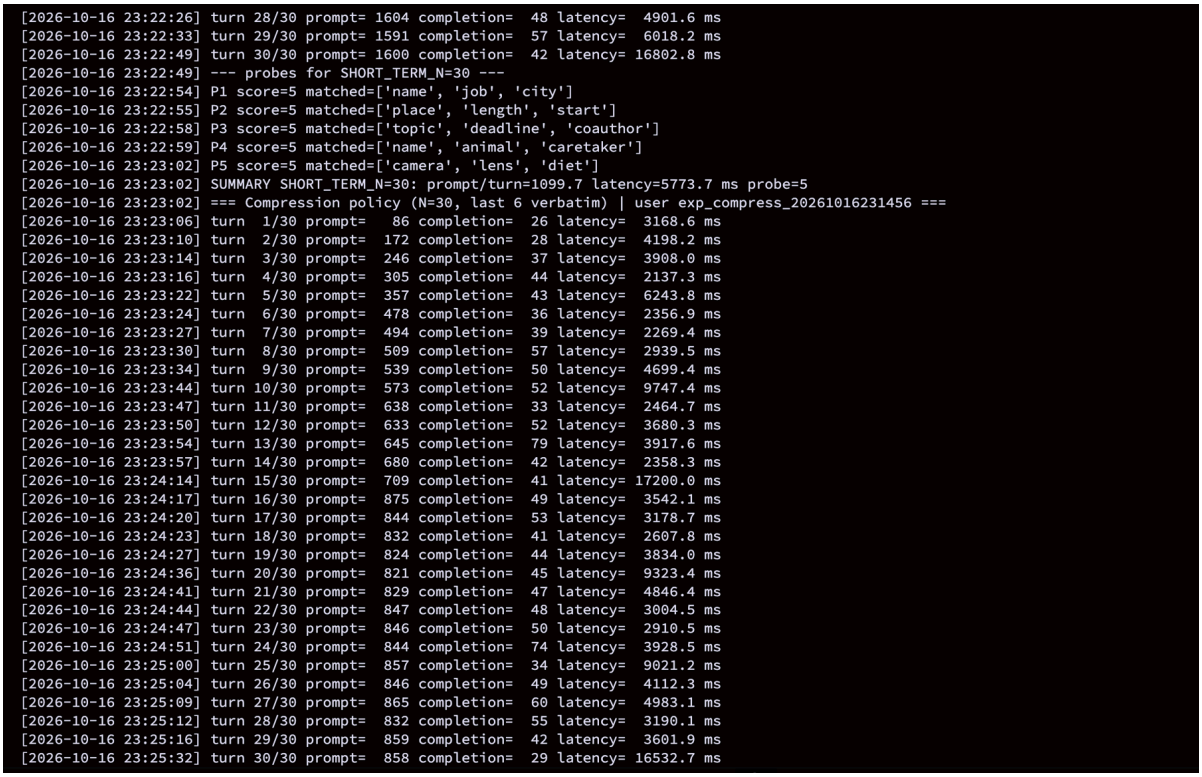
30 turns with tokens and latency, then the five probe scores.

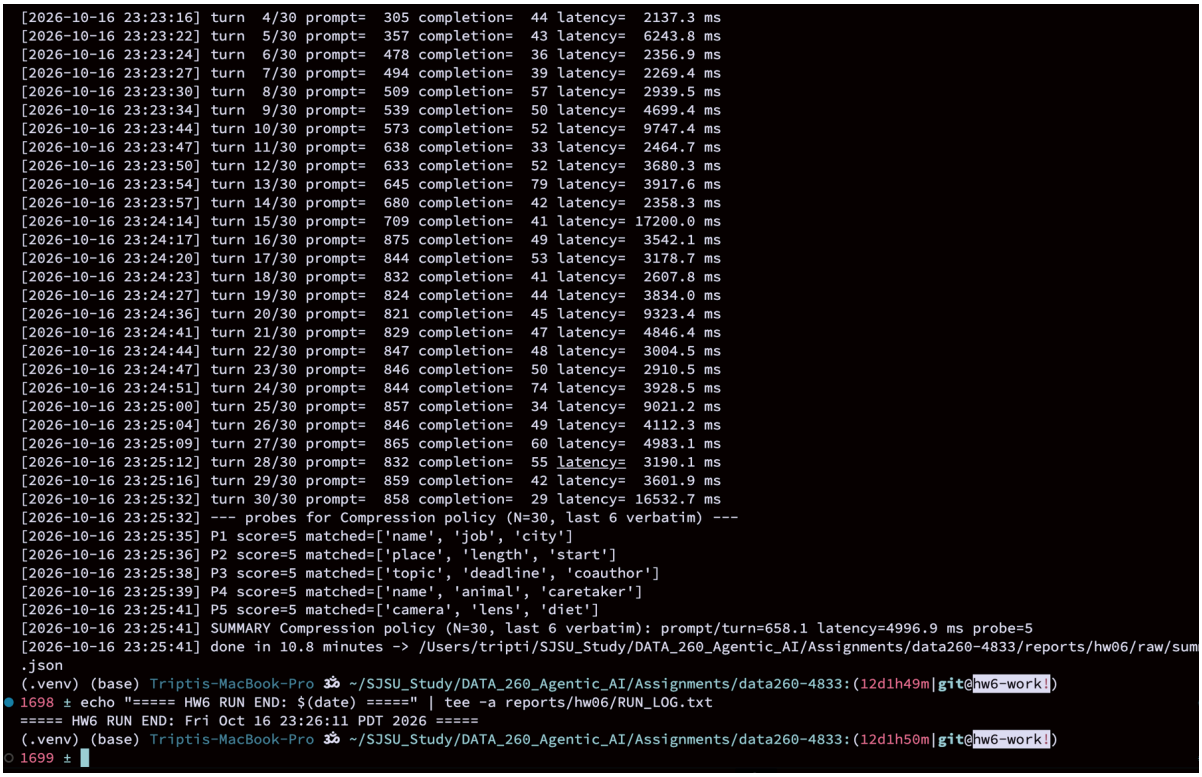
```
===== HW6 RUN START: Fri Oct 16 23:14:47 PDT 2026 =====
(.venv) (base) Triptis-MacBook-Pro ~/SJSU_Study/DATA_260_Agentic_AI/Assignments/data260-4833:(12d1h39m|git@hw6-work)
1697 ± caffeinate -i python -u run_hw06_memory_experiment.py --configs n4,n12,n30,compress 2>&1 | tee -a reports/hw06/RUN_LOG.txt
[2026-10-16 23:14:56] === SHORT_TERM_N=4 | user exp_n4_20261016231456 ===
[2026-10-16 23:15:01] turn  1/30 prompt=   86 completion=  26 latency=  5634.9 ms
[2026-10-16 23:15:06] turn  2/30 prompt=  172 completion=  28 latency=  4398.4 ms
[2026-10-16 23:15:10] turn  3/30 prompt=  246 completion=  37 latency=  4082.2 ms
[2026-10-16 23:15:12] turn  4/30 prompt=  252 completion=  41 latency=  2201.0 ms
[2026-10-16 23:15:19] turn  5/30 prompt=  264 completion=  47 latency=  6598.1 ms
[2026-10-16 23:15:21] turn  6/30 prompt=  352 completion=  34 latency=  2102.1 ms
[2026-10-16 23:15:23] turn  7/30 prompt=  349 completion=  44 latency=  2273.3 ms
[2026-10-16 23:15:25] turn  8/30 prompt=  347 completion=  44 latency=  2218.6 ms
[2026-10-16 23:15:30] turn  9/30 prompt=  358 completion=  51 latency=  4570.3 ms
[2026-10-16 23:15:39] turn 10/30 prompt=  368 completion=  42 latency=  9179.5 ms
[2026-10-16 23:15:41] turn 11/30 prompt=  405 completion=  28 latency=  1988.9 ms
[2026-10-16 23:15:44] turn 12/30 prompt=  379 completion=  36 latency=  2752.6 ms
[2026-10-16 23:15:46] turn 13/30 prompt=  366 completion=  57 latency=  2762.0 ms
[2026-10-16 23:15:49] turn 14/30 prompt=  391 completion=  38 latency=  2076.6 ms
[2026-10-16 23:16:05] turn 15/30 prompt=  398 completion=  38 latency= 16294.9 ms
[2026-10-16 23:16:08] turn 16/30 prompt=  528 completion=  46 latency=  2962.3 ms
[2026-10-16 23:16:10] turn 17/30 prompt=  542 completion=  50 latency=  2580.2 ms
[2026-10-16 23:16:12] turn 18/30 prompt=  536 completion=  34 latency=  1924.9 ms
[2026-10-16 23:16:16] turn 19/30 prompt=  531 completion=  45 latency=  3367.6 ms
[2026-10-16 23:16:23] turn 20/30 prompt=  529 completion=  50 latency=  7428.5 ms
[2026-10-16 23:16:27] turn 21/30 prompt=  538 completion=  42 latency=  4209.2 ms
[2026-10-16 23:16:30] turn 22/30 prompt=  547 completion=  43 latency=  2219.4 ms
[2026-10-16 23:16:32] turn 23/30 prompt=  539 completion=  49 latency=  2482.2 ms
[2026-10-16 23:16:35] turn 24/30 prompt=  531 completion=  49 latency=  2493.8 ms
[2026-10-16 23:16:42] turn 25/30 prompt=  530 completion=  34 latency=  7626.7 ms
[2026-10-16 23:16:46] turn 26/30 prompt=  541 completion=  45 latency=  3854.9 ms
[2026-10-16 23:16:50] turn 27/30 prompt=  558 completion=  41 latency=  3656.6 ms
[2026-10-16 23:16:52] turn 28/30 prompt=  553 completion=  38 latency=  2092.6 ms
[2026-10-16 23:16:55] turn 29/30 prompt=  543 completion=  46 latency=  3398.1 ms
[2026-10-16 23:17:08] turn 30/30 prompt=  550 completion=  31 latency= 13077.8 ms
[2026-10-16 23:17:08] --- probes for SHORT_TERM_N=4 ---
[2026-10-16 23:17:10] P1 score=5 matched=['name', 'job', 'city']
[2026-10-16 23:17:11] P2 score=5 matched=['place', 'length', 'start']
[2026-10-16 23:17:13] P3 score=5 matched=['topic', 'deadline', 'coauthor']
[2026-10-16 23:17:14] P4 score=1 matched=[]
[2026-10-16 23:17:16] P5 score=5 matched=['camera', 'lens', 'diet']
[2026-10-16 23:17:16] SUMMARY SHORT_TERM_N=4: prompt/turn=427.6 latency=4416.9 ms probe=4.2
```

Output 3 - Probe answers and scores (SHORT_TERM_N=4)

The model's answers and the score for each probe.

```
[2026-10-16 23:19:31] turn 29/30 prompt=  856 completion=  37 latency=  3482.6 ms
[2026-10-16 23:19:46] turn 30/30 prompt=  844 completion=  38 latency= 15405.0 ms
[2026-10-16 23:19:46] --- probes for SHORT_TERM_N=12 ---
[2026-10-16 23:19:49] P1 score=5 matched=['name', 'job', 'city']
[2026-10-16 23:19:50] P2 score=5 matched=['place', 'length', 'start']
[2026-10-16 23:19:52] P3 score=5 matched=['topic', 'deadline', 'coauthor']
[2026-10-16 23:19:54] P4 score=4 matched=['name', 'animal']
[2026-10-16 23:19:56] P5 score=5 matched=['camera', 'lens', 'diet']
[2026-10-16 23:19:56] SUMMARY SHORT_TERM_N=12: prompt/turn=676.7 latency=4997.4 ms probe=4.8
[2026-10-16 23:19:56] === SHORT_TERM_N=30 | user exp_n30_20261016231456 ===
[2026-10-16 23:19:59] turn  1/30 prompt=   86 completion=  26 latency=  3282.4 ms
[2026-10-16 23:20:03] turn  2/30 prompt=  172 completion=  28 latency=  4200.4 ms
[2026-10-16 23:20:07] turn  3/30 prompt=  246 completion=  37 latency=  3828.6 ms
[2026-10-16 23:20:09] turn  4/30 prompt=  305 completion=  44 latency=  2197.2 ms
[2026-10-16 23:20:16] turn  5/30 prompt=  381 completion=  41 latency=  6054.4 ms
[2026-10-16 23:20:18] turn  6/30 prompt=  537 completion=  49 latency=  2375.0 ms
[2026-10-16 23:20:20] turn  7/30 prompt=  611 completion=  48 latency=  2429.3 ms
[2026-10-16 23:20:23] turn  8/30 prompt=  687 completion=  54 latency=  2628.0 ms
[2026-10-16 23:20:28] turn  9/30 prompt=  765 completion=  55 latency=  4677.3 ms
[2026-10-16 23:20:36] turn 10/30 prompt=  848 completion=  40 latency=  8513.3 ms
[2026-10-16 23:20:40] turn 11/30 prompt=  941 completion=  40 latency=  3335.0 ms
[2026-10-16 23:20:42] turn 12/30 prompt= 1008 completion=  38 latency=  2837.8 ms
[2026-10-16 23:20:46] turn 13/30 prompt= 1064 completion=  67 latency=  3329.0 ms
[2026-10-16 23:20:48] turn 14/30 prompt= 1147 completion=  54 latency=  2770.0 ms
[2026-10-16 23:21:05] turn 15/30 prompt= 1229 completion=  37 latency= 16486.2 ms
[2026-10-16 23:21:09] turn 16/30 prompt= 1447 completion=  48 latency=  4471.6 ms
[2026-10-16 23:21:14] turn 17/30 prompt= 1468 completion=  52 latency=  4716.3 ms
[2026-10-16 23:21:19] turn 18/30 prompt= 1469 completion=  50 latency=  4552.0 ms
[2026-10-16 23:21:25] turn 19/30 prompt= 1474 completion=  59 latency=  5968.6 ms
[2026-10-16 23:21:35] turn 20/30 prompt= 1491 completion=  51 latency= 10178.9 ms
[2026-10-16 23:21:42] turn 21/30 prompt= 1520 completion=  60 latency=  6886.9 ms
[2026-10-16 23:21:47] turn 22/30 prompt= 1543 completion=  51 latency=  5119.8 ms
[2026-10-16 23:21:52] turn 23/30 prompt= 1541 completion=  50 latency=  4830.8 ms
[2026-10-16 23:21:58] turn 24/30 prompt= 1530 completion=  82 latency=  6015.1 ms
[2026-10-16 23:22:08] turn 25/30 prompt= 1543 completion=  46 latency= 10140.7 ms
[2026-10-16 23:22:14] turn 26/30 prompt= 1552 completion=  58 latency=  6416.1 ms
[2026-10-16 23:22:22] turn 27/30 prompt= 1590 completion=  71 latency=  7248.2 ms
[2026-10-16 23:22:26] turn 28/30 prompt= 1604 completion=  48 latency=  4901.6 ms
[2026-10-16 23:22:33] turn 29/30 prompt= 1591 completion=  57 latency=  6018.2 ms
[2026-10-16 23:22:49] turn 30/30 prompt= 1600 completion=  42 latency= 16802.8 ms
[2026-10-16 23:22:49] --- probes for SHORT_TERM_N=30 ---
```





Results (generated from reports/hw06/raw/summary.json):

Setting	Mean prompt tokens/turn	Mean latency (ms)	Probe quality (1-5)
SHORT_TERM_N=4	427.6	4416.9	4.2
SHORT_TERM_N=12	676.7	4997.4	4.8
SHORT_TERM_N=30	1099.7	5773.7	5

Compression policy (N=30, last 6 verbatim)	658.1	4996.9	5

Where the prompt tokens go (mean per turn):

Setting	Primer	Long-term	Short-term	Episodic
SHORT_TERM_N=4	50.4	186.2	132.3	42.8
SHORT_TERM_N=12	49.1	179.6	391.3	41.4
SHORT_TERM_N=30	48.9	192.0	802.2	41.3
Compression policy (N=30, last 6 verbatim)	49.2	201.4	350.6	41.4

3.4 Memory-Saving Policy:

I tested one compression policy on top of N = 30. The last 6 messages stay word for word. For the older messages in the window, I drop the assistant replies and keep each older user message as one short bullet. Then I ran the same conversation and probes again.

Code 2 - Compression policy (hw6/memory.py)

digest_older drops assistant replies and shortens user messages to 120 characters; apply_policy keeps the last 6 messages word for word.

```
267
268
269 def digest_older(messages: List[Dict[str, Any]], max_chars: int = 120) -> str:
270     """Compress older messages: drop assistant replies, shorten user lines to bullets."""
271     lines = []
272     for message in messages:
273         if message["role"] != "user":
274             continue
275         text = " ".join(message["content"].split())
276         if len(text) > max_chars:
277             text = text[: max_chars - 3].rstrip() + "..."
278         lines.append(f"- {text}")
279     return "\n".join(lines)
280
281
282 def apply_policy(window: List[Dict[str, Any]], cfg: MemoryConfig) -> Tuple[str, List[Dict[str, Any]]]:
283     """Return (digest_text, verbatim_messages) for the chosen memory-saving policy."""
284     if cfg.policy != "compress_older" or len(window) <= cfg.keep_recent:
285         return "", window
286     older = window[: -cfg.keep_recent]
287     recent = window[-cfg.keep_recent :]
288     return digest_older(older), recent
289
290
```

Measure	N=30 (baseline)	N=30 + compression	Change
Mean prompt tokens/turn	1099.7	658.1	-441.6 (-40.2%)
Probe quality (1-5)	5	5	+0.00
Mean latency (ms)	5773.7	4996.9	-776.8

Output 4 - Compression comparison

Token savings, probe quality and latency before and after.



3.5 Memory Cost Estimate:

I used $1 per million input tokens and $5 per million output tokens, for 10,000 conversations of 30 turns each. The token counts include the reply call and the background calls (fact extraction and summaries).

Setting	Input tokens	Output tokens	Total cost
SHORT_TERM_N=4	214,760,000	27,340,000	$351.46
SHORT_TERM_N=12	291,060,000	29,210,000	$437.11
SHORT_TERM_N=30	419,390,000	30,250,000	$570.64
Compression policy (N=30, last 6 verbatim)	287,950,000	30,470,000	$440.30

Best setting: SHORT_TERM_N=12. **Worst setting:** SHORT_TERM_N=4.

I chose them with a rule I fixed before the runs. Best is the highest mean probe score; if settings are within 0.2 points of the top, I take the cheapest of them. Worst is the lowest mean probe score; if settings are that close, I take the most expensive. The cost difference between the best and worst setting is $85.65 per 10,000 conversations.

Output 5 - Final metrics tables

Tables generated from the raw files by make_hw06_metrics.py.

```
(.venv) (base) Triptis-MacBook-Pro ~/SJSU_Study/DATA_260_Agentic_AI/Assignments/data260-4833:(12d1h56m|git@hw6-work!)
1700 ± echo "Trapti Kulshrestha | SID4 4833"
python make_hw06_metrics.py
Trapti Kulshrestha | SID4 4833
# HW6 Metrics (generated by make_hw06_metrics.py from reports/hw06/raw/)

## Table 1 - Memory settings (30-message conversation, 5 probes)

| Setting | Mean prompt tokens/turn | Mean latency (ms) | Probe quality (1-5) |
|---|---|---|---|
| SHORT_TERM_N=4 | 427.6 | 4416.9 | 4.2 |
| SHORT_TERM_N=12 | 676.7 | 4997.4 | 4.8 |
| SHORT_TERM_N=30 | 1099.7 | 5773.7 | 5 |
| Compression policy (N=30, last 6 verbatim) | 658.1 | 4996.9 | 5 |

## Table 2 - Where the prompt tokens go (mean per turn)

Part sizes are the real prompt token count split in proportion to each part's text length.

| Setting | System primer | Long-term summaries | Short-term window | Episodic facts | Current message |
|---|---|---|---|---|---|
| SHORT_TERM_N=4 | 50.4 | 186.2 | 132.3 | 42.8 | 15.9 |
| SHORT_TERM_N=12 | 49.1 | 179.6 | 391.3 | 41.4 | 15.4 |
| SHORT_TERM_N=30 | 48.9 | 192.0 | 802.2 | 41.3 | 15.3 |
| Compression policy (N=30, last 6 verbatim) | 49.2 | 201.4 | 350.6 | 41.4 | 15.5 |

## Table 3 - Probe scores per question

| Setting | P1 | P2 | P3 | P4 | P5 | Mean |
|---|---|---|---|---|---|---|
| SHORT_TERM_N=4 | 5 | 5 | 5 | 1 | 5 | 4.2 |
| SHORT_TERM_N=12 | 5 | 5 | 5 | 4 | 5 | 4.8 |
| SHORT_TERM_N=30 | 5 | 5 | 5 | 5 | 5 | 5 |
| Compression policy (N=30, last 6 verbatim) | 5 | 5 | 5 | 5 | 5 | 5 |

## Table 4 - Compression policy vs SHORT_TERM_N=30

Policy: keep the last 6 messages word for word; for older messages in the window, drop assistant replies and keep only shortened user mes
es as bullets.

| Measure | N=30 (before) | Compression (after) | Change |
|---|---|---|---|
| Mean prompt tokens/turn | 1099.7 | 658.1 | -441.6 (-40.2%) |
| Probe quality (1-5) | 5 | 5 | +0.00 |
| Mean latency (ms) | 5773.7 | 4996.9 | -776.8 ms |

## Table 5 - Cost of 10,000 conversations x 30 turns

Pricing: input $1 per million tokens, output $5 per million tokens. Tokens include the reply call and the background calls (episode extra
on and summaries).

| Setting | Input tokens | Output tokens | Input cost | Output cost | Total cost |
|---|---|---|---|---|---|
| SHORT_TERM_N=4 | 214,760,000 | 27,340,000 | $214.76 | $136.70 | $351.46 |
| SHORT_TERM_N=12 | 291,060,000 | 29,210,000 | $291.06 | $146.05 | $437.11 |
| SHORT_TERM_N=30 | 419,390,000 | 30,250,000 | $419.39 | $151.25 | $570.64 |
| Compression policy (N=30, last 6 verbatim) | 287,950,000 | 30,470,000 | $287.95 | $152.35 | $440.30 |

Best setting: **SHORT_TERM_N=30**. Worst setting: **SHORT_TERM_N=4**.

How they were chosen: best = highest mean probe score, and if several settings are within 0.2 points of the top, the cheapest of them. Wo
 = lowest mean probe score, and if several are within the same margin, the most expensive of them. The rule was fixed before the runs and
 in make_hw06_metrics.py.

Wrote /Users/tripti/SJSU_Study/DATA_260_Agentic_AI/Assignments/data260-4833/reports/hw06/METRICS.md
(.venv) (base) Triptis-MacBook-Pro ~/SJSU_Study/DATA_260_Agentic_AI/Assignments/data260-4833:(12d1h57m|git@hw6-work!)
1701 ±
```

3.6 Results and Conclusion:

Going from N=4 to N=12 to N=30 changed the mean prompt tokens per turn from 427.6 to 676.7 to 1099.7, and the mean probe quality from 4.2 to 4.8 to 5. The mean latency went from 4416.9 ms to 4997.4 ms to 5773.7 ms.

The compression policy used 441.6 fewer prompt tokens per turn than plain N=30 (40.2% less). Probe quality went from 5 to 5, and latency from 5773.7 ms to 4996.9 ms.

This is one run per setting on a local 8B model, so small differences can be noise. All numbers come from reports/hw06/raw/ and METRICS.md.

I did not expect the 4-message window to forget the cat. The fact was stored in MongoDB, but the top-3 episodic retrieval did not bring it back. Based on my predefined selection rule, SHORT_TERM_N=12 was the best overall setting because its probe score was within 0.2 of the maximum while having the lowest cost among those settings. The compression policy was also effective because it kept a perfect probe score of 5 while reducing prompt tokens by about 40% compared with plain N=30.

Part 4 – File-Based Memory

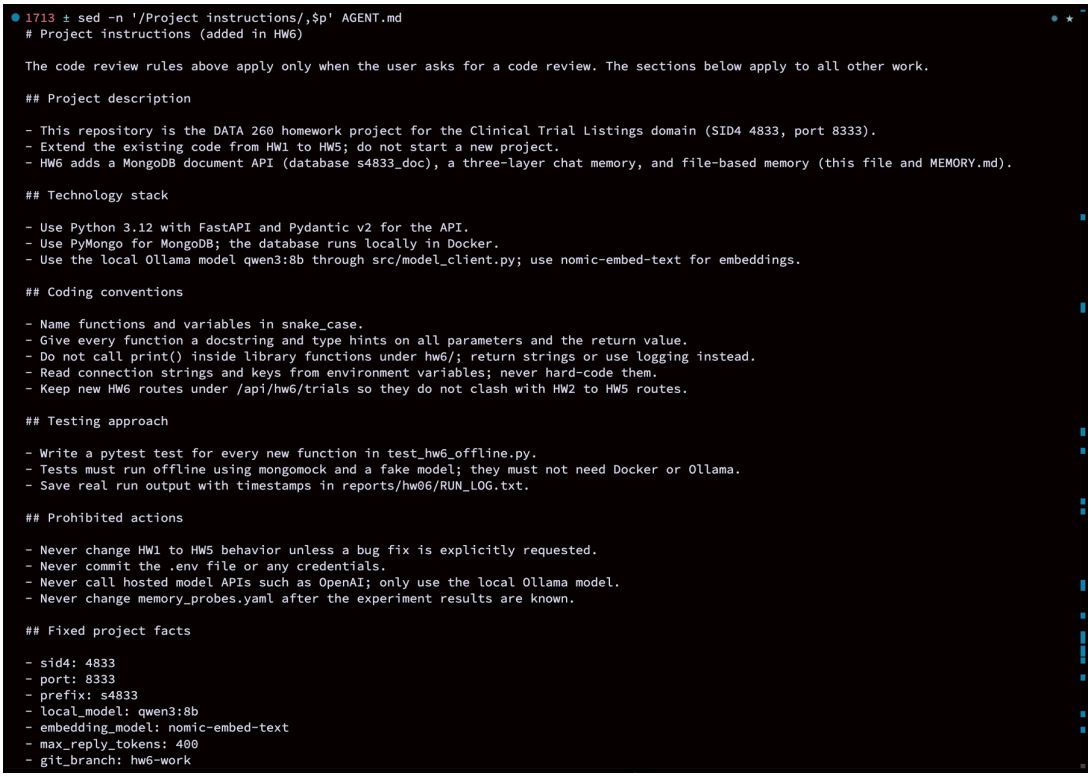
The second memory system uses two Markdown files in the repository root. AGENT.md stores durable instructions and MEMORY.md stores durable facts. Both are loaded into the system prompt of a small command-line chat (file_memory_chat.py).

4.1 AGENT.md:

I kept the original code review rules from HW1 and added a new section with 18 instructions in five categories: project description, technology stack, coding conventions, testing approach and prohibited actions. It also has a short list of fixed project facts. The consolidation step uses these facts to check its own decisions.

Output 1 - AGENT.md instructions (HW6 section)

Five categories, 18 instructions, and the fixed project facts.



```
1713 ± sed -n '/Project instructions/,$p' AGENT.md
# Project instructions (added in HW6)

The code review rules above apply only when the user asks for a code review. The sections below apply to all other work.

## Project description

- This repository is the DATA 260 homework project for the Clinical Trial Listings domain (SID4 4833, port 8333).
- Extend the existing code from HW1 to HW5; do not start a new project.
- HW6 adds a MongoDB document API (database s4833_doc), a three-layer chat memory, and file-based memory (this file and MEMORY.md).

## Technology stack

- Use Python 3.12 with FastAPI and Pydantic v2 for the API.
- Use PyMongo for MongoDB; the database runs locally in Docker.
- Use the local Ollama model qwen3:8b through src/model_client.py; use nomic-embed-text for embeddings.

## Coding conventions

- Name functions and variables in snake_case.
- Give every function a docstring and type hints on all parameters and the return value.
- Do not call print() inside library functions under hw6/; return strings or use logging instead.
- Read connection strings and keys from environment variables; never hard-code them.
- Keep new HW6 routes under /api/hw6/trials so they do not clash with HW2 to HW5 routes.

## Testing approach

- Write a pytest test for every new function in test_hw6_offline.py.
- Tests must run offline using mongomock and a fake model; they must not need Docker or Ollama.
- Save real run output with timestamps in reports/hw06/RUN_LOG.txt.

## Prohibited actions

- Never change HW1 to HW5 behavior unless a bug fix is explicitly requested.
- Never commit the .env file or any credentials.
- Never call hosted model APIs such as OpenAI; only use the local Ollama model.
- Never change memory_probes.yaml after the experiment results are known.

## Fixed project facts

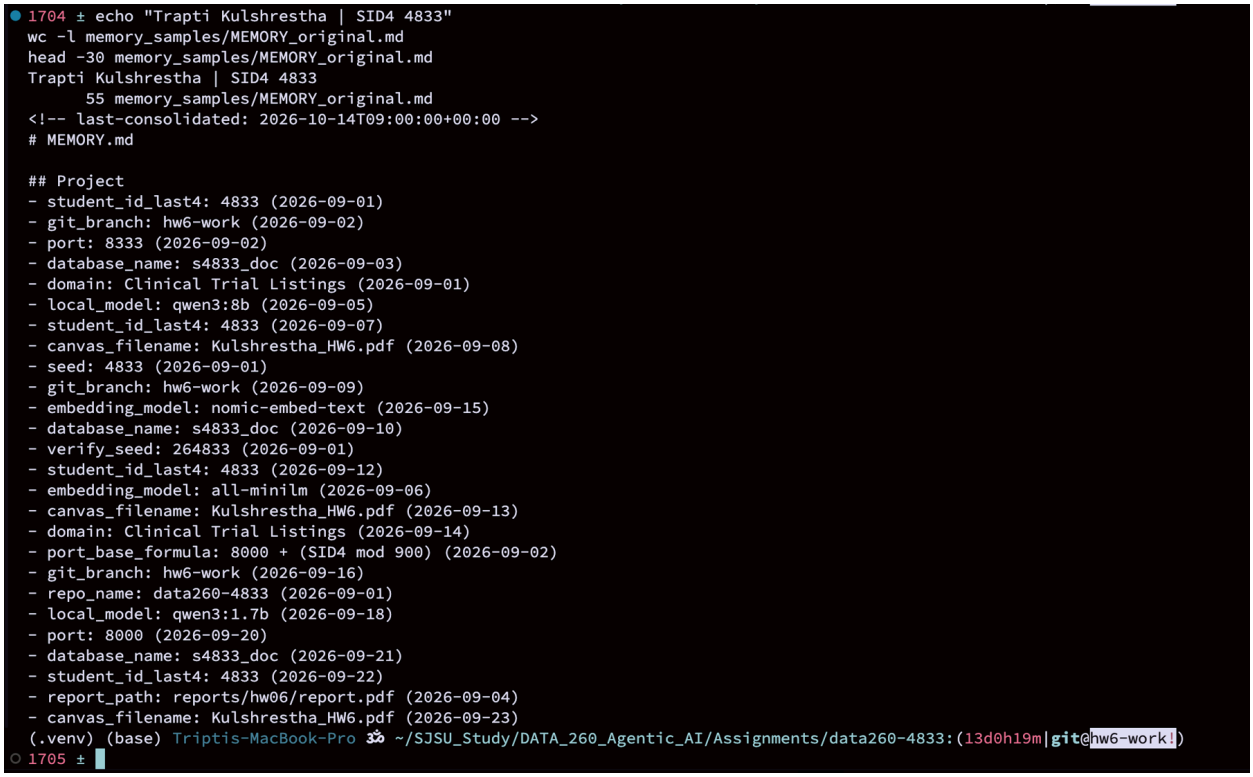
- sid4: 4833
- port: 8333
- prefix: s4833
- local_model: qwen3:8b
- embedding_model: nomic-embed-text
- max_reply_tokens: 400
- git_branch: hw6-work
```

4.2 MEMORY.md:

MEMORY.md stores facts as key: value (date). The first line is a comment with the last-consolidated time. For the consolidation test, I created a 55-line version with repeated facts and conflicting facts on purpose.

Output 2 - Original MEMORY.md (55 lines)

Start of the original file, with duplicates and conflicting values.



4.3 File Loading and Consolidation Checks:

The code loads both files and joins them into the system prompt. It also reads the last-consolidated time and decides if consolidation is required: more than 40 non-empty, non-comment lines, or a last consolidation more than 24 hours ago.

4.4 /memory:

The /memory command shows the AGENT.md sections, the number of active lines, whether consolidation is needed, and the MEMORY.md text.

Code 1 - Command handler (hw6/file_memory.py)

/help, /memory, /compact and /dream.

```python
def handle_command(
    command: str,
    history: List[Dict[str, str]],
    chat_fn: ChatFunction,
    memory_path: Path = config.MEMORY_FILE,
    agent_path: Path = config.AGENT_FILE,
    now: Optional[datetime] = None,
) -> str:
    """Run /memory, /compact, /dream or /help and return the text to show."""
    name = command.strip().split()[0].lower() if command.strip() else ""
    if name == "/help":
        return HELP_TEXT
    if name == "/memory":
        memory_text = load_memory_md(memory_path)
        required, reasons = needs_consolidation(memory_text, now)
        status = "consolidation needed: " + ("yes (" + "; ".join(reasons) + ")" if required else "no")
        return (
            f"AGENT.md sections:\n{summarize_agent_md(load_agent_md(agent_path))}\n\n"
            f"MEMORY.md ({count_active_lines(memory_text)} active lines, {status}):\n{memory_text}"
        )
    if name == "/compact":
        result = compact_file(memory_path, agent_path, now)
        return "MEMORY.md consolidated.\n" + format_compact_report(result)
    if name == "/dream":
        updated, added = dream(history, load_memory_md(memory_path), chat_fn, now.date() if now else None)
        if not added:
            return "Nothing new worth remembering from the recent conversation."
        Path(memory_path).write_text(updated, encoding="utf-8")
        return "Added to MEMORY.md:\n" + "\n".join(added)
    return f"Unknown command {name}. Type /help."

```

Output 3 - /memory

Active line count and the reason consolidation is needed.

```
1716 ± python -c "from pathlib import Path; from datetime import datetime, timezone; from hw6 import file_memory as f; print(f.handle_comman
d('/memory', [], None, Path('memory_samples/MEMORY_original.md'), Path('AGENT.md'), datetime.now(timezone.utc)))" | head -16
AGENT.md sections:
  Project description: 3 lines
  Technology stack: 3 lines
  Coding conventions: 5 lines
  Testing approach: 3 lines
  Prohibited actions: 4 lines
  Fixed project facts: 7 lines

MEMORY.md (52 active lines, consolidation needed: yes (52 active lines (limit 40); last consolidation 92.8 hours ago (limit 24))):
<!-- last-consolidated: 2026-10-14T09:00:00+00:00 -->
# MEMORY.md

## Project
- student_id_last4: 4833 (2026-09-01)
- git_branch: hw6-work (2026-09-02)
- port: 8333 (2026-09-02)
(.venv) (base) Triptis-MacBook-Pro 🐼 ~/SJSU_Study/DATA_260_Agentic_AI/Assignments/data260-4833:(13d1h13m|git@hw6-work!)
1717 ±
```

4.5 /compact:

The /compact command consolidates MEMORY.md and reports the number of entries and lines before and after.

Output 4 - /compact before and after

Entries and lines before and after, merged duplicates and resolved contradictions.

```
1705 ± echo "Trapti Kulshrestha | SID4 4833"
python file_memory_chat.py --once /compact
Trapti Kulshrestha | SID4 4833
File-based memory chat | model qwen3:8b | type /help for commands, /quit to exit
MEMORY.md consolidated.
Entries before: 49   after: 27
Lines before:   55   after: 39
Duplicate facts merged: 7
  - student_id_last4: 4833 (5 copies -> 1)
  - git_branch: hw6-work (4 copies -> 1)
  - database_name: s4833_doc (3 copies -> 1)
  - domain: Clinical Trial Listings (3 copies -> 1)
  - canvas_filename: Kulshrestha_HW6.pdf (3 copies -> 1)
  - test_command: pytest -q test_hw6_offline.py (5 copies -> 1)
  - compression_policy: compress_older (2 copies -> 1)
Contradictions resolved: 4
  - port: kept "8333" (recency rule picked "8000" but AGENT.md says "8333", so it was corrected)
  - local_model: kept "qwen3:8b" (recency rule picked "qwen3:1.7b" but AGENT.md says "qwen3:8b", so it was corrected)
  - embedding_model: kept "nomic-embed-text" (most recent value wins)
  - max_reply_tokens: kept "400" (most recent value wins)
Resolved incorrectly by the recency rule: 2
  - port: first chose "8000" - recency rule picked "8000" but AGENT.md says "8333", so it was corrected
  - local_model: first chose "qwen3:1.7b" - recency rule picked "qwen3:1.7b" but AGENT.md says "qwen3:8b", so it was corrected
(.venv) (base) Triptis-MacBook-Pro 🕉 ~/SJSU_Study/DATA_260_Agentic_AI/Assignments/data260-4833:(13d0h20m|git@hw6-work!)
1706 ±
```

4.6 /dream:

The /dream command reads the recent conversation, asks the model for up to 5 long-term facts, and adds the new ones to MEMORY.md. Exact repeats are skipped.

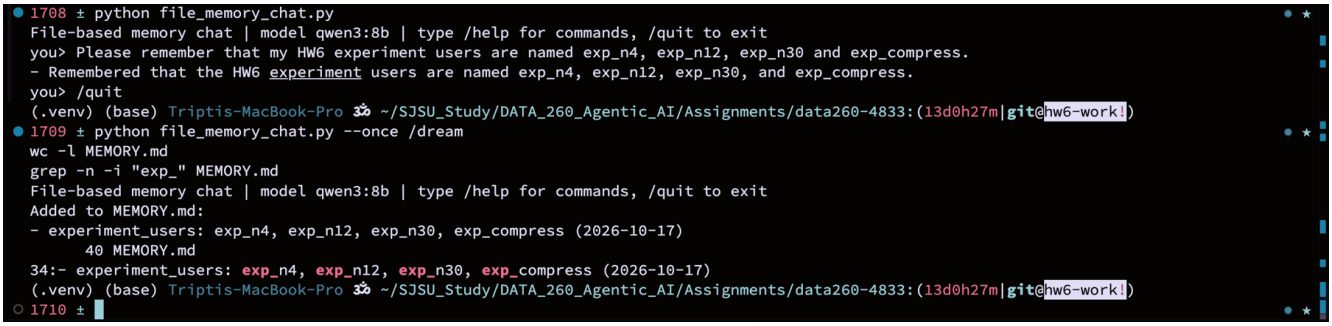
Code 2 - /dream (hw6/file_memory.py)

Fact extraction from recent chat and append to MEMORY.md.

```python
def dream(
    conversation: List[Dict[str, str]],
    memory_text: str,
    chat_fn: ChatFunction,
    today: Optional[date] = None,
) -> Tuple[str, List[str]]:
    """Extract long-term facts from recent conversation; return (new_memory_text, added_lines)."""
    existing, _ = parse_entries(memory_text)
    known_keys = ", ".join(sorted({item.key for item in existing})) or "(none)"
    transcript = "\n".join(f"{m['role']}: {m['content']}" for m in conversation[-20:])
    reply = chat_fn(
        [
            {"role": "system", "content": DREAM_PROMPT},
            {"role": "user", "content": f"Existing keys: {known_keys}\n\nConversation:\n{transcript}"},
        ]
    )
    facts: List[Tuple[str, str]] = []
    for item in parse_json_array(reply)[:5]:
        if isinstance(item, dict) and isinstance(item.get("key"), str) and item.get("value") is not None:
            key = re.sub(r"\s+", "_", item["key"].strip().lower())
            value = str(item["value"]).strip()
            if key and value:
                facts.append((key, value))
    return append_facts(memory_text, facts, today)
```

Output 5 - /dream

New facts added to MEMORY.md.

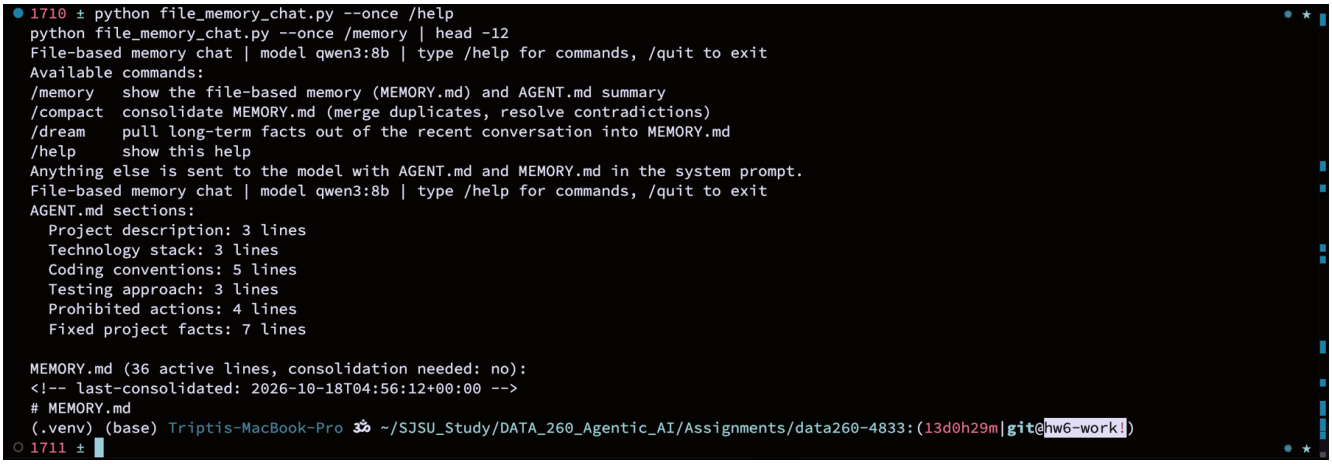


4.7 /help:

The /help command explains the four commands.

Output 6 - /help

List of the available commands.



4.8 Consolidation Test:

The original file has 55 lines and 49 fact lines. After consolidation it has 39 lines and 27 facts. 7 duplicated facts were merged and 4 contradictions were resolved.

For each contradiction, the newest value wins and the older value is written into a Resolved contradictions section, so nothing is silently deleted. Then each result is checked against the fixed facts in AGENT.md. A resolution that disagrees with AGENT.md is marked as incorrect and corrected, and the line says so.

Key	Kept	Dropped	Rule	Correct?
port	8333	8000	newest date wins, then checked against AGENT.md	No - recency chose "8000"; corrected
local_model	qwen3:8b	qwen3:1.7b	newest date wins, then checked against AGENT.md	No - recency chose "qwen3:1.7b"; corrected
embedding_model	nomic-embed-text	all-minilm	newest date wins, then checked against	Yes

			AGENT.md	
max_reply_tokens	400	200	newest date wins, then checked against AGENT.md	Yes

Examples of merged duplicates: student_id_last4 (5 copies), git_branch (4 copies), database_name (3 copies), domain (3 copies).

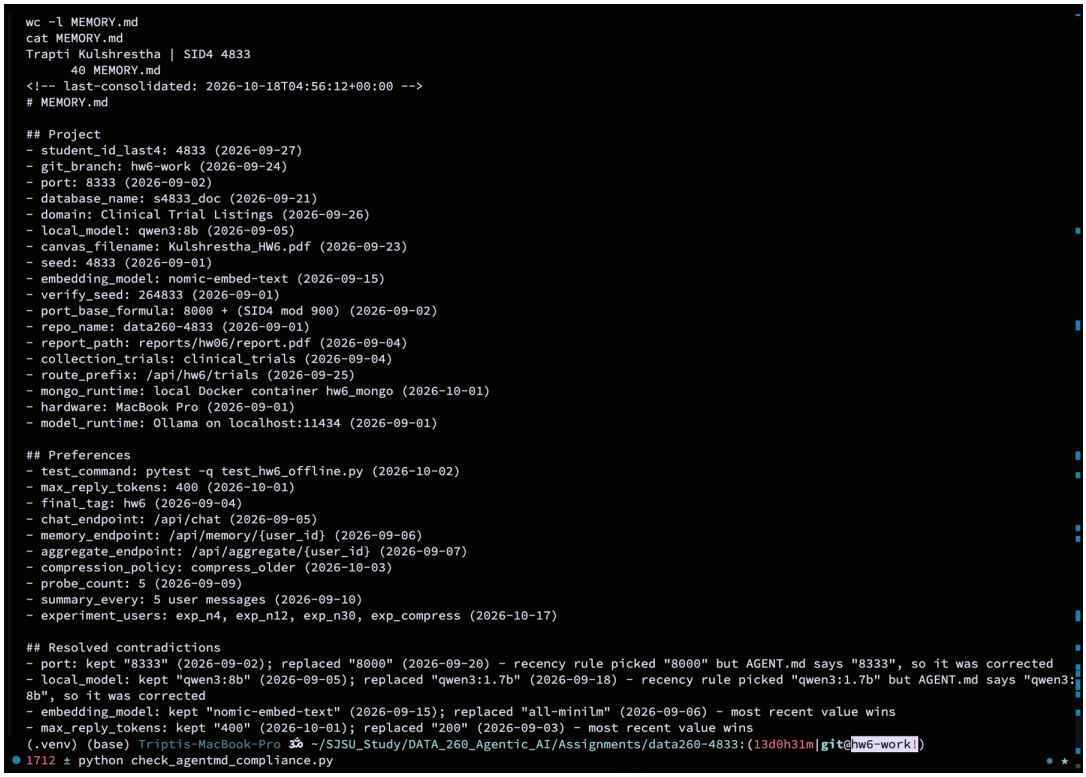
Code 3 - Contradiction resolution (hw6/file_memory.py)

Newest wins, then a check against AGENT.md.

```python
def resolve_group(key: str, entries: List[Entry], fixed: Dict[str, str]) -> Dict[str, object]:
    """Resolve one key that has more than one distinct value."""
    by_value: Dict[str, List[Entry]] = {}
    for entry in entries:
        by_value.setdefault(normalize(entry.value), []).append(entry)
    candidates = {value: pick_latest(items) for value, items in by_value.items()}
    winner = pick_latest(list(candidates.values()))
    first_pass = winner
    fixed_value = fixed.get(normalize(key))
    wrong = False
    note = "most recent value wins"
    if fixed_value is not None and normalize(winner.value) != normalize(fixed_value):
        wrong = True
        corrected = candidates.get(normalize(fixed_value))
        if corrected is not None:
            winner = corrected
            note = f'recency rule picked "{first_pass.value}" but AGENT.md says "{fixed_value}", so it was corrected'
        else:
            note = f'NEEDS REVIEW: AGENT.md says "{fixed_value}" but no stored value matches'
    rejected = [item for item in candidates.values() if item is not winner]
    return {
        "key": key,
        "winner": winner,
        "rejected": rejected,
        "first_pass_value": first_pass.value,
        "first_pass_wrong": wrong,
```

Output 7 - Consolidated MEMORY.md (under 40 lines)

Merged facts and an explicit Resolved contradictions section.



Both versions are committed to Git: the original first, then the consolidated version.

4.9 AGENT.md Compliance Test:

I picked one function, count_active_lines(), and checked it against the AGENT.md coding rules with check_agentmd_compliance.py. The checker verifies the snake_case name, the docstring, type hints on every parameter and the return value, no print() call, no hard-coded secret, and a pytest test for the function. I also ran the checker on a deliberately bad function to confirm that it fails when a rule is broken.

Code 4 - Function checked (hw6/file_memory.py)

count_active_lines() with type hints and a docstring.

```
69
70  def count_active_lines(text: str) -> int:
71      """Count non-empty lines that are not inside HTML comments."""
72      count = 0
73      in_comment = False
74      for raw_line in text.splitlines():
75          line = raw_line.strip()
76          if in_comment:
77              if "-->" in line:
78                  in_comment = False
79              continue
80          if not line:
81              continue
82          if line.startswith("<!--"):
83              if "-->" not in line:
84                  in_comment = True
85              continue
86          count += 1
87      return count
88
```

Output 8 - AGENT.md compliance check

All rule checks pass for the function.



4.10 MongoDB Memory vs File-Based Memory:

Criterion	MongoDB memory (Parts 2-3)	File-based memory (AGENT.md, MEMORY.md)
Inspectability	Needs Compass or a query. The embedding vectors are not readable.	Open the file in any editor. Every line is readable and shows up in Git diffs.
Persistence after restart	Persists in the Docker volume. It is lost if the volume is deleted.	Persists as normal files and is versioned in Git.
Scalability across sessions	Good. Indexed queries and top-k search work for many users and sessions.	Poor. The whole file goes into every prompt, so it must stay small.
Facts that must not be forgotten	Risky. A fact can fall out of the window or never be retrieved.	Better. The facts are always in the prompt, and old values are kept in a log. The newest-wins rule can still pick wrongly, so a check against AGENT.md is needed.

Best use: MongoDB for per-user conversation memory at scale; MEMORY.md and AGENT.md for a small set of project rules and facts that must always be in the prompt.

Verification

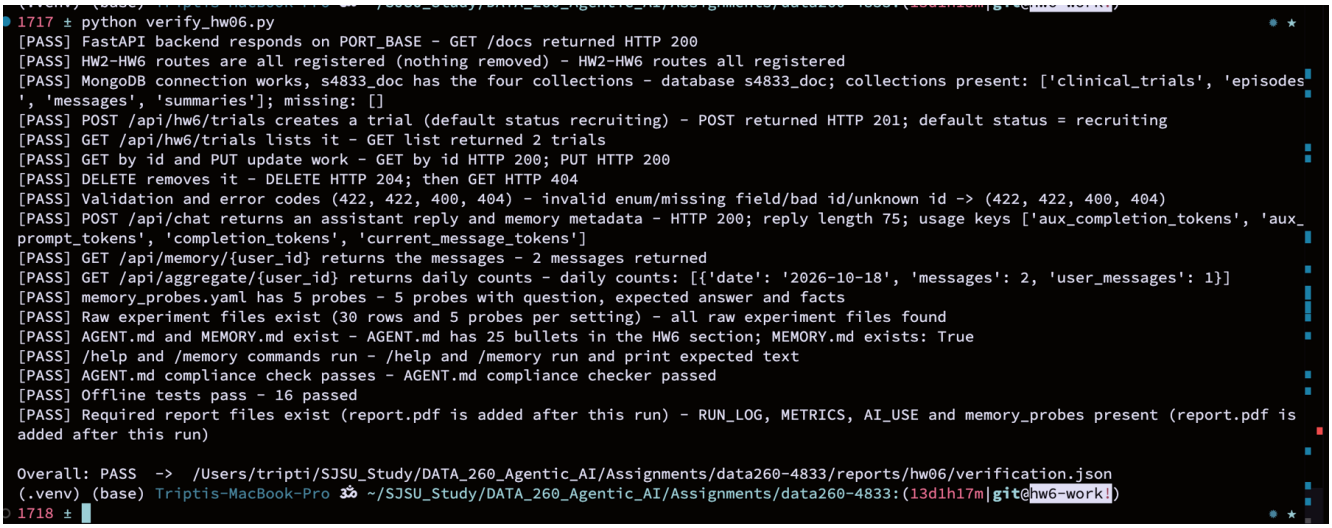
I ran verify_hw06.py. It ran 18 checks and the overall result was PASS.

Check	Result
FastAPI backend responds on PORT_BASE	PASS
HW2-HW6 routes are all registered (nothing removed)	PASS
MongoDB connection works, s4833_doc has the four collections	PASS
POST /api/hw6/trials creates a trial (default status recruiting)	PASS
GET /api/hw6/trials lists it	PASS
GET by id and PUT update work	PASS
DELETE removes it	PASS
Validation and error codes (422, 422, 400, 404)	PASS
POST /api/chat returns an assistant reply and memory metadata	PASS

GET /api/memory/{user_id} returns the messages	PASS
GET /api/aggregate/{user_id} returns daily counts	PASS
memory_probes.yaml has 5 probes	PASS
Raw experiment files exist (30 rows and 5 probes per setting)	PASS
AGENT.md and MEMORY.md exist	PASS
/help and /memory commands run	PASS
AGENT.md compliance check passes	PASS
Offline tests pass	PASS
Required report files exist (report.pdf is added after this run)	PASS

Output 1 - verify_hw06.py

Smoke test result.



AI Use

1. What I used AI for and what I did myself

I used Claude and ChatGPT to understand the HW6 requirements and to plan the order of the work. I set up the branch and folders, ran Docker, MongoDB and Ollama on my own machine, tested the code, reviewed the results, ran every experiment, took every screenshot, and did all the Git work myself.

2. An AI suggestion that was unsuitable

The model call used qwen3:8b with its default settings. When I ran the chat endpoint for real, many replies came back empty. The code looked correct and passed the offline tests, which use a fake model, so the problem only showed up with the real model.

3. How I detected the problem

I noticed the empty replies while running the chat experiments. I then called the model directly with a short diagnostic. It used up its tokens (174) on hidden thinking and returned no visible text. The fake model in the offline tests never does this, which is why the tests did not catch it.

4. What I changed and why it works now

I turned off the hidden thinking for qwen3 (reasoning=False) in hw6/llm.py and added one retry with a larger token budget in case a reply is still empty. I also changed the fact extraction prompt so it keeps names. After this the replies were no longer empty, and I re-ran the tests and all experiments. Every number in the report comes from those real runs.